

claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf\_a8f43370-4cb\agent-ad1cd0ea4421883e2.jsonl`
- SHA-256: `43d0dee1da2446fdab45661a48cc9d3ddc822d35e0871d6ea6235c3388869d5e`
- Source modified: `2026-07-07T15:17:08+00:00`
- Imported at: `2026-07-09T08:32:24+00:00`
- project: `wf\_a8f43370-4cb`
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`

Transcript

1. user (2026-07-07T15:07:26.824Z)

You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic "WASB weights not found at /opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):

- BACKEND (repo undefined, branch undefined, PR #undefined):
backend/orchestration.py adds `_WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid, fusion_hybrid}` and `_local_detector_weights_present(cfg, seg_name)`. In `_maybe_dispatch_remote` the choice=="local" branch now fails fast via `_failed(msg, False)` when the segmenter is weights-backed AND `_cloud_available(cfg)` AND local weights are not readable; otherwise returns None (in-process as before). Plus 4 tests in `tests/test_api_cloud_dispatch.py`.
- SPA (repo undefined, branch undefined, PR #undefined):
web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when `capabilities.deployment.compute_target=="cloud_run"` (else keeps the free "local" default). Plus 1 test.

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:

- undefined/backend.diff (backend commit diff)
- undefined/orchestration_new.py (FULL post-change backend/orchestration.py)
- undefined/test_api_cloud_dispatch_new.py (FULL post-change tests)
- undefined/spa.diff (SPA commit diff)
- undefined/IngestPanel_new.tsx (FULL post-change IngestPanel.tsx)

You MAY read other repo files for ground truth, e.g. under undefined:

backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;
backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;
backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);
and under undefined: web/src/api/types.ts, web/src/api/client.ts,
web/src/components/IngestPanel.test.tsx.

You may also run: `gh pr diff undefined -R Khelsutra/badminton-highlight-indexer` and `gh pr diff undefined -R avidullu/khelsutra`.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete config/inputs

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.

If nothing real exists in your dimension, return an empty findings array.

=== YOUR DIMENSION: guardrail-correctness ===

CORE LOGIC of the backend guardrail in `_maybe_dispatch_remote` (choice=="local" branch).

Check: does it fire EXACTLY when intended and never otherwise? Any bypass (e.g.

casing/whitespace of

`seg_name` or `req.compute`)? Does the guardrail sit correctly relative to the later choice=="cloud"

branch and the default branch ? no fallback that re-runs in-process? Is `_failed(msg, False)`

correct here (prune of partial rows via `play_wm/cand_wm`; provenance path=`not_run`,

`requested=local`,

`target=cloud_run`, `dispatched=False`)? Does the segmenter get instantiated before the guardrail returns

(it must NOT)? Any ordering/short-circuit bug in the boolean AND? Does the `reingest/cleanup`

finally

block behave correctly for this early return?

2. assistant (2026-07-07T15:07:43.167Z)

I'll start by exploring the environment to locate the authoritative snapshot files and the repo structure, since the task uses "undefined" placeholders for paths.

3. user (2026-07-07T15:07:45.914Z)

=== cwd ===

/c/Users/avidu/Projects/khelsutra-guru/khelsutra

=== working dir listing ===

total 62

drwxr-xr-x	1	avidu	197609	0	Jul	5	12:57	.
drwxr-xr-x	1	avidu	197609	0	Jul	7	19:57	..
drwxr-xr-x	1	avidu	197609	0	Jun	22	11:12	.claude
drwxr-xr-x	1	avidu	197609	0	Jul	7	20:33	.git
-rw-r--r--	1	avidu	197609	297	Jun	20	23:27	.gitattributes
drwxr-xr-x	1	avidu	197609	0	Jun	20	12:43	.github
-rw-r--r--	1	avidu	197609	323	Jun	22	14:57	.gitignore
drwxr-xr-x	1	avidu	197609	0	Jul	2	21:13	deploy
drwxr-xr-x	1	avidu	197609	0	Jul	2	21:13	docs
drwxr-xr-x	1	avidu	197609	0	Jun	21	23:55	locales
-rw-r--r--	1	avidu	197609	9773	Jul	2	21:13	NEXT_STEPS.md
drwxr-xr-x	1	avidu	197609	0	Jun	21	15:38	node_modules
-rw-r--r--	1	avidu	197609	1469	Jun	23	00:19	README.md
drwxr-xr-x	1	avidu	197609	0	Jun	23	00:19	site
drwxr-xr-x	1	avidu	197609	0	Jul	7	19:45	web

=== parent ===

total 108

drwxr-xr-x	1	avidu	197609	0	Jul	7	19:57	.
drwxr-xr-x	1	avidu	197609	0	Jul	7	20:24	..
drwxr-xr-x	1	avidu	197609	0	Jun	23	09:53	Annotation Setup
-rw-r--r--	1	avidu	197609	905	Jul	2	21:20	AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x	1	avidu	197609	0	Jul	7	20:14	badminton-highlight-indexer
-rw-r--r--	1	avidu	197609	1222	Jul	2	21:35	bash.exe.stackdump
drwxr-xr-x	1	avidu	197609	0	Jul	7	20:34	bhi-wt-worker-reval
drwxr-xr-x	1	avidu	197609	0	Jul	5	12:57	khelsutra
drwxr-xr-x	1	avidu	197609	0	Jul	5	18:47	khelsutra-screenshot
drwxr-xr-x	1	avidu	197609	0	Jul	5	13:44	marketing
drwxr-xr-x	1	avidu	197609	0	Jul	5	13:43	pr-158-screenshots
drwxr-xr-x	1	avidu	197609	0	Jun	22	15:03	rally-annotator
drwxr-xr-x	1	avidu	197609	0	Jul	4	06:58	sports-data-collector
drwxr-xr-x	1	avidu	197609	0	Jul	2	19:46	sports-obsreport

4. user (2026-07-07T15:07:47.610Z)

=== search for snapshot files ===

5. assistant (2026-07-07T15:07:57.468Z)

The literal snapshot files (with "undefined" prefix) weren't found. Let me search more broadly and locate the actual backend/SPA repos.

6. user (2026-07-07T15:08:03.981Z)

=== backend repo top-level ===

```
total 742
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 25 23:31 .claude
-rw-r--r-- 1 avidu 197609 94208 Jul  7 20:14 .coverage
-rw-r--r-- 1 avidu 197609   611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609    69 Jun 10 23:25 .env
-rw-r--r-- 1 avidu 197609 1059 Jul  7 18:29 .gcloudignore
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:35 .git
-rw-r--r-- 1 avidu 197609   947 Jun 25 16:24 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:13 .github
-rw-r--r-- 1 avidu 197609 1998 Jul  2 08:13 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 .vscode
drwxr-xr-x 1 avidu 197609    0 Jul  7 10:27 __pycache__
drwxr-xr-x 1 avidu 197609    0 Jul  4 00:22 artifacts
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:56 backend
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 backend.db
-rw-r--r-- 1 avidu 197609   6619 Jul  2 11:50 CLAUDE.md
-rw-r--r-- 1 avidu 197609   2735 Jun 16 01:52 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:17 docs
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 eval_baselines
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 indexer.db
-rw-r--r-- 1 avidu 197609   1999 Jun 25 16:24 mypy.ini
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 output
-rw-r--r-- 1 avidu 197609   5413 Jul  4 13:58 pyproject.toml
-rw-r--r-- 1 avidu 197609    240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 rally_index.db
-rw-r--r-- 1 avidu 197609 35511 Jul  2 08:13 README.md
-rw-r--r-- 1 avidu 197609   1674 Jul  2 20:05 requirements.txt
-rw-r--r-- 1 avidu 197609    585 Jul  2 19:49 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609    809 Jun 25 16:24 ruff.toml
-rwxr-xr-x 1 avidu 197609   2118 Jun 29 13:47 run_all_tests.py
-rw-r--r-- 1 avidu 197609   5739 Jun 29 13:47 run_eval.py
-rw-r--r-- 1 avidu 197609   4594 Jun 29 13:47 run_regression.py
-rwxr-xr-x 1 avidu 197609   5073 Jul  7 10:27 score_golden_gate.py
-rwxr-xr-x 1 avidu 197609   7617 Jul  7 03:32 score_local.py
```

=== bhi-wt-worker-reval (worktree?) ===

```
total 362
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:34 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
-rw-r--r-- 2 avidu 197609 94208 Jul  7 20:34 .coverage
-rw-r--r-- 1 avidu 197609   611 Jul  7 19:57 .coveragerc
-rw-r--r-- 1 avidu 197609 1059 Jul  7 19:57 .gcloudignore
-rw-r--r-- 1 avidu 197609   110 Jul  7 19:57 .git
-rw-r--r-- 1 avidu 197609   947 Jul  7 19:57 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 .github
-rw-r--r-- 1 avidu 197609 1998 Jul  7 19:57 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:08 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:06 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:06 .ruff_cache
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 .vscode
```

```

drwxr-xr-x 1 avidu 197609      0 Jul  7 20:09 __pycache__
drwxr-xr-x 1 avidu 197609      0 Jul  7 20:05 backend
-rw-r--r-- 1 avidu 197609 6619 Jul  7 19:57 CLAUDE.md
-rw-r--r-- 1 avidu 197609 2735 Jul  7 19:57 config.json
drwxr-xr-x 1 avidu 197609      0 Jul  7 19:57 deploy
drwxr-xr-x 1 avidu 197609      0 Jul  7 19:57 docs
drwxr-xr-x 1 avidu 197609      0 Jul  7 19:57 eval_baselines
-rw-r--r-- 1 avidu 197609 1999 Jul  7 19:57 mypy.ini
drwxr-xr-x 1 avidu 197609      0 Jul  7 20:34 output
-rw-r--r-- 1 avidu 197609 5413 Jul  7 19:57 pyproject.toml
-rw-r--r-- 1 avidu 197609  245 Jul  7 19:57 pytest.ini
-rw-r--r-- 1 avidu 197609 35511 Jul  7 19:57 README.md
-rw-r--r-- 1 avidu 197609 1674 Jul  7 19:57 requirements.txt
-rw-r--r-- 1 avidu 197609  585 Jul  7 19:57 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609  809 Jul  7 19:57 ruff.toml
-rwxr-xr-x 1 avidu 197609 2118 Jul  7 19:57 run_all_tests.py
-rw-r--r-- 1 avidu 197609 5739 Jul  7 19:57 run_eval.py
-rw-r--r-- 1 avidu 197609 4594 Jul  7 19:57 run_regression.py
drwxr-xr-x 1 avidu 197609      0 Jul  7 20:09 scripts
-rw-r--r-- 1 avidu 197609 2410 Jul  7 19:57 SESSION_HANDOFF.md
drwxr-xr-x 1 avidu 197609      0 Jul  7 20:28 tests
drwxr-xr-x 1 avidu 197609      0 Jul  7 20:09 tools
drwxr-xr-x 1 avidu 197609      0 Jul  7 20:09 training

```

7. user (2026-07-07T15:08:07.781Z)

```

=== search whole projects tree + temp for snapshot files ===
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/backend.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/IngestPanel_new.tsx
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_new.py
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/spa.diff
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/test_api_cloud_dispatch_new.py
=== find orchestration.py in backend repo ===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py

```

8. assistant (2026-07-07T15:08:15.766Z)

Found the snapshots in the scratchpad. Let me read the authoritative backend files (my dimension is the backend guardrail).

9. user (2026-07-07T15:08:16.765Z)

```

1      commit 4565c7436ccf0b4e30f68f01efe6082eff371a5d
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date:   Tue Jul 7 19:56:22 2026 +0530
4
5          fix(serving): fail fast when compute=local forces weightless in-process detection
6
7          A hosted cloud-serving control-plane bakes NO detector weights into its image
8          (only the worker Cloud Run Job GCSFuse-mounts them at /gcs/models/...). So a
9          POST /api/process with compute="local" for a weights-backed trajectory detector
10         (wasb_hybrid / tracknet_hybrid / fusion_hybrid) returned None from
11         _maybe_dispatch_remote and ran the in-process segmenter ON THE CONTROL-PLANE,
12         which then died deep in the healthcheck with the confusing
13         "WASB weights not found at /opt/models/WASB-SBDT/..." (observed live 2026-07-07,
14         job f29d41d0?: [COMPUTE] requested=local actual=in_process ... status=failed).
15
16         Guardrail: on a cloud-serving-capable server (_cloud_available) whose local
17         weights for the requested detector aren't readable, a forced compute="local"

```

```

18     now fails FAST with an actionable message ("this hosted server can't run
19     detection locally ... Choose 'Run in cloud'.") ? mirroring the existing
20     compute="cloud"-not-available reject ? instead of the silent in-process trap.
21     A truly-local box, or a cloud-wired box that actually has the weights on disk,
22     is unaffected (the weights-present check is the escape hatch); a weight-free
23     segmenter (motion) is out of scope. Honest provenance: path=not_run,
24     requested=local, dispatched=False.
25
26     Covered by tests/test_api_cloud_dispatch.py (fail-fast, weights-present escape
27     hatch, motion unaffected, pure-local not guardrailed).
28
29     Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
30
31     diff --git a/backend/orchestration.py b/backend/orchestration.py
32     index 5e9f510..bfe39f5 100644
33     --- a/backend/orchestration.py
34     +++ b/backend/orchestration.py
35     @@ -155,6 +155,38 @@ def _ingest_remote_segments(
36         return n
37
38
39     + # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS
40     box. A hosted
41     + # cloud-serving control-plane bakes NO detector weights into its image (only the worker
42     Cloud Run Job
43     + # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
44     succeed ? the
45     + # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
46     deep
47     + # "WASB weights not found". (Names are the registered segmenter identities; see
48     backend/pipeline/segmenters.)
49     + _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
50     "fusion_hybrid"})
51     +
52     +
53     + def _local_detector_weights_present(cfg, seg_name: str) -> bool:
54     +     """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
55     present + readable
56     +     on THIS box. Mirrors the native/WSL runners' resolution (env override ?
57     ``indexing.<det>.weights_path``)
58     +     and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
59     that would
60     +     otherwise fail deep in the run. Fusion follows its configured substrate (WASB
61     default / TrackNet).
62     +     A non-weights segmenter returns True (nothing local to check)."""
63     +     name = (seg_name or "").strip().lower()
64     +     if name not in _WEIGHTS_BACKED_SEGMENTERS:
65     +         return True
66     +     idx = getattr(cfg, "indexing", None)
67     +     substrate = (
68     +         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
69     +     ).strip().lower()
70     +     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
71     "tracknet"):
72     +         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
73     +     else: # wasb_hybrid, or fusion on the (default) WASB substrate
74     +         path = (
75     +             os.environ.get("WASB_WEIGHTS")
76     +             or getattr(getattr(idx, "wasb", None), "weights_path", "")
77     +             or ""
78     +         )
79     +     return bool(path) and os.path.exists(os.path.expanduser(path))
80
81     def _cloud_available(cfg) -> bool:

```

```

72         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
73 3).
74     @@ -446,6 +478,23 @@ def _maybe_dispatch_remote(
75         )
76         choice = ""
77         if choice == "local":
78             + # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
79             + # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
80             + # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
81             + # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail
FAST with an
82             + # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
83             + # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it
runs in-process
84             + # below exactly as before (the weights-present check is the escape hatch).
85             + if (
86             +     (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
87             +     and _cloud_available(cfg)
88             +     and not _local_detector_weights_present(cfg, seg_name)
89             + ):
90             +     return _failed(
91             +         f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
92             +         "live only on the cloud worker, not on the control-plane. Choose 'Run
in cloud'.",
93             +         False,
94             +     )
95             return None # explicit "run on my machine" ? in-process regardless of the
server's target
96         if choice == "cloud":
97             if not _cloud_available(cfg):
98                 diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
99                 index 8382234..7993b7f 100644
100                 --- a/tests/test_api_cloud_dispatch.py
101                 +++ b/tests/test_api_cloud_dispatch.py
102                 @@ -721,6 +721,114 @@ def
test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypa
103                     assert all(_meta(s)["source"] == "cloud_run" for s in segs)
104
105
106             + # --- hosted control-plane guardrail: compute='local' for a weights-backed detector
-----
107             + # A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
Run Job mounts
108             + # them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
there. It must fail
109             + # FAST with an actionable "run in the cloud" message ? not fall through to the in-
process segmenter
110             + # that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
2026-07-07).
111             +
112             +
113             + def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
114             +     """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
115             +     a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
found')."""
116             +     _db, _bucket, _mp = cloud_env
117             +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
118             +     m.global_config.indexing.wasb.weights_path =

```

```

"/nonexistent/wasb_badminton_best.pth.tar"
119 +     fake = _FakeCloudClient()
120 +     _inject_fake(monkeypatch, fake)
121 +
122 +     out = m._execute_processing(
123 +         m.ProcessRequest(
124 +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
125 +         )
126 +     )
127 +     assert out["status"] == "failed"
128 +     assert "can't run detection locally" in out["message"]
129 +     assert "Run in cloud" in out["message"]
130 +     assert "weights not found" not in out["message"] # NOT the cryptic deep failure
131 +     assert fake.calls == [] # never dispatched
132 +     # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
133 +     assert out["compute"]["requested"] == "local"
134 +     assert out["compute"]["dispatched"] is False
135 +     assert out["compute"]["path"] == "not_run"
136 +
137 +
138 +def test_local_on_cloud_server_with_weights_present_runs_in_process(
139 +    cloud_env, tmp_path, monkeypatch
140 +):
141 +    """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
142 +    ACTUALLY has the
143 +    detector weights on disk, compute='local' still runs the in-process segmenter (no
144 +    dispatch)."""
145 +    _db, _bucket, _mp = cloud_env
146 +    weights = tmp_path / "wasb_badminton_best.pth.tar"
147 +    weights.write_bytes(b"WEIGHTS")
148 +    monkeypatch.delenv("WASB_WEIGHTS", raising=False)
149 +    m.global_config.indexing.wasb.weights_path = str(weights)
150 +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
151 +    fake = _FakeCloudClient()
152 +    _inject_fake(monkeypatch, fake)
153 +
154 +    out = m._execute_processing(
155 +        m.ProcessRequest(
156 +            video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
157 +        )
158 +    )
159 +    assert out["status"] == "success"
160 +    assert fake.calls == [] # in-process, not dispatched
161 +    assert out["compute"]["path"] == "in_process"
162 +
163 +
164 +def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
165 +    cloud_env, monkeypatch
166 +):
167 +    """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
168 +    (motion) with
169 +    compute='local' on a cloud server is unaffected ? it runs in-process, never fails
170 +    fast."""
171 +    _db, _bucket, _mp = cloud_env
172 +    m.global_config.indexing.wasb.weights_path =
173 +    "/nonexistent/should_not_matter.pth.tar"
174 +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
175 +    fake = _FakeCloudClient()
176 +    _inject_fake(monkeypatch, fake)
177 +
178 +    out = m._execute_processing(
179 +        m.ProcessRequest(
180 +            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
181 +        )
182 +    )

```

```

178 +     assert out["status"] == "success"
179 +     assert fake.calls == []
180 +     assert out["compute"]["path"] == "in_process"
181 +
182 +
183 +def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
184 +     tmp_path, monkeypatch
185 +):
186 +     """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
187 +     weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
188 +     'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
189 +     db = Database(str(tmp_path / "t.db")) # noqa: F841
190 +     monkeypatch.setattr(m, "db_instance", db)
191 +     cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}}) # noqa:
F841
192 +     monkeypatch.setattr(m, "global_config", cfg)
193 +     monkeypatch.setattr(
194 +         m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
195 +     )
196 +     monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
197 +     monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
198 +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
199 +     cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
200 +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
201 +     video = tmp_path / "v.mp4"
202 +     video.write_bytes(b"hashable-bytes")
203 +
204 +     out = m._execute_processing(
205 +         m.ProcessRequest(
206 +             video_path=str(video), segmenter="wasb_hybrid", compute="local"
207 +         )
208 +     )
209 +     # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
210 +     assert out["status"] == "success"
211 +     assert out["compute"]["path"] == "in_process"
212 +
213 +
214 +     # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----
215
216
217

```

10. user (2026-07-07T15:08:17.692Z)

```

1     """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3     The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4     Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5     and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6     Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8     SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9     monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),

```



```

never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
43         from backend.main import ProcessRequest # noqa: F401
44
45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
48     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
61         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62         """
63         if segments:

```

```

64         db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65     else:
66         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69     def _validation_failure_message(failures: List[Any]) -> str:
70         details = []
71         for failure in failures:
72             name = getattr(failure, "validator_name", None) or "validation"
73             message = (getattr(failure, "message", None) or "").strip()
74             details.append(f"{name}: {message}" if message else str(name))
75         if not details:
76             return "Video failed validation checks."
77         return "Video failed validation: " + "; ".join(details)
78
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
82         scratch."""
83         storage_config = (
84             config.get("storage")
85             if isinstance(config, dict)
86             else getattr(config, "storage", None)
87         )
88         try:
89             if storage.input_is_local(raw_input, storage_config):
90                 return config
91         except ValueError:
92             return config
93         try:
94             cfg = (
95                 config.model_dump(mode="json")
96                 if hasattr(config, "model_dump")
97                 else dict(config or {})
98             )
99         except Exception:
100             cfg = {}
101         report = dict(cfg.get("report") or {})
102         if report.get("output_dir"):
103             return config
104         report["output_dir"] = os.path.join(output_base(config), "reports")
105         cfg["report"] = report
106         return cfg
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
112         the bucket and
113         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
114         call ? so
115         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
116         were produced.
117         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
118         pattern (cloud
119         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
120         malformed row is
121         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
122         temp dir is
123         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
124         a job failure
125         and prunes any partial rows (destroy-AFTER-confirm)."""
126         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
127         ref = storage.parse_uri(uri)

```

```

121     n = 0
122     with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
123         local = storage.fetch(
124             uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
125         )
126         with open(local, newline="", encoding="utf-8") as f:
127             for row in csv.DictReader(f):
128                 if n >= max_rows:
129                     logger.warning(
130                         "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
131                         max_rows,
132                         uri,
133                     )
134                     break
135                 try:
136                     start, end = float(row["start"]), float(row["end"])
137                 except (KeyError, TypeError, ValueError):
138                     continue
139                 if not (math.isfinite(start) and math.isfinite(end)):
140                     continue # reject inf/nan times rather than persist a junk rally
141                 meta: Dict[str, Any] = {
142                     "source": "cloud_run",
143                     "ending_reason": (row.get("ending_reason") or "").strip(),
144                 }
145                 sc = (row.get("shot_count") or "").strip()
146                 if sc:
147                     try:
148                         meta["shot_count"] = int(
149                             float(sc)
150                         ) # OverflowError on 'inf'/'1e400'
151                     except (ValueError, OverflowError):
152                         meta["shot_count"] = sc
153                 db.add_play_segment(video_id, start, end, metadata=meta)
154                 n += 1
155     return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:
174             return True

```

```

175     idx = getattr(cfg, "indexing", None)
176     substrate = (
177         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178     ).strip().lower()
179     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181     else: # wasb_hybrid, or fusion on the (default) WASB substrate
182         path = (
183             os.environ.get("WASB_WEIGHTS")
184             or getattr(getattr(idx, "wasb", None), "weights_path", "")
185             or ""
186         )
187     return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190 def _cloud_available(cfg) -> bool:
191     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
192
193     True when the control plane is wired for Cloud Run: either the configured default
target is
194     already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195     (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196     are present so a forced per-request override can actually dispatch + collect a
result.
197
198     Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
199     deployment = getattr(cfg, "deployment", None)
200     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201         return True
202     has_job = bool(
203         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204     )
205     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206     return bool(has_job and out_root)
207
208
209 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
211
212     with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213     (queue/cold-start), so the very first tick already shows movement."""
214     minutes = int(elapsed_sec) // 60
215     if minutes < 1:
216         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222     object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223     deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224     ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225     if getattr(input_ref, "backend", "") != "gcs":
226         return None
227     sc = _storage_settings(cfg)

```

```

228         if not sc:
229             return None
230         try:
231             st = storage.get_storage("gcs", config=sc).stat(input_ref)
232             return getattr(st, "md5", None) or None
233     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234         logger.info(
235             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236             getattr(input_ref, "uri", input_ref),
237             exc_info=True,
238         )
239     return None
240
241
242     def _cached_process_payload(
243         video_id: str, validation_results, segments, requested_compute
244     ) -> Dict[str, Any]:
245         """The /api/process result for a video whose detection already exists ? ONE shape
246         for
247         both producers (the drain's DB-cache short-circuit and the submit-time cache,
248         #484)."""
249         return {
250             "video_id": video_id,
251             "status": "success",
252             "message": (
253                 f"Reused {len(segments)} cached play segments. "
254                 "Pass force=true to reprocess this video."
255             ),
256             "results": validation_results or [],
257             "play_segments": segments,
258             "cached": True,
259             "compute": {"path": "cached", "requested": requested_compute},
260         }
261
262     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
263         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
264         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
265         decide."""
266         ref = storage.parse_uri(uri)
267         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
268             local = storage.fetch(
269                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
270             )
271             with open(local, encoding="utf-8") as f:
272                 loaded = json.load(f)
273             return loaded if isinstance(loaded, dict) else None
274
275     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
276     hits
277     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
278     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
279     instance
280     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
281     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
282     _REHYDRATE_LOCKS_GUARD = threading.Lock()
283
284     def _rehydrate_lock(video_id: str) -> threading.Lock:
285         with _REHYDRATE_LOCKS_GUARD:
286             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
287
288     def probe_process_cache(db, cfg, video_id: str) -> bool:

```

```

288     """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
289
290     Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
291     ordering). Used to decide whether a submit can skip the local-scratch fetch guard
292     (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
293     errors report False (? the normal dispatch path with its guards)."""
294     try:
295         existing = db.get_video(video_id)
296         if (
297             existing
298             and existing.get("status") == "processed"
299             and db.query_play_segments(video_id, {})
300         ):
301             return True
302         sc = _storage_settings(cfg)
303         if not sc or (sc.get("backend") or "local") == "local":
304             return False
305         output_root = (sc.get("output_root") or "").rstrip("/")
306         if not output_root:
307             return False
308         report_uri = f"{output_root}/outputs/{video_id}/report.json"
309         if not storage.exists(report_uri, config=sc):
310             return False
311         report = _read_bucket_json(report_uri, sc)
312         return bool(report and report.get("status") == "success")
313     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
314         logger.warning(
315             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
316         )
317         return False
318
319
320     def cached_process_result(
321         db, cfg, video_id: str, video_path: str, requested_compute=None
322     ) -> Optional[Dict[str, Any]]:
323         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
324         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
325         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
326         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
327         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
328         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
329         partial earlier ingest is replaced, never double-counted.
330
331         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
332         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
333         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
334         job before invoking (writes happen post-claim)."""
335         try:
336             with _rehydrate_lock(video_id):
337                 existing = db.get_video(video_id)
338                 if existing and existing.get("status") == "processed":
339                     segs = db.query_play_segments(video_id, {})
340                     if segs:
341                         return _cached_process_payload(
342                             video_id,

```

```

343         existing.get("validation_results", []),
344         segs,
345         requested_compute,
346     )
347     sc = _storage_settings(cfg)
348     if not sc or (sc.get("backend") or "local") == "local":
349         return None
350     output_root = (sc.get("output_root") or "").rstrip("/")
351     if not output_root:
352         return None
353     outputs_uri = f"{output_root}/outputs/{video_id}"
354     report_uri = f"{outputs_uri}/report.json"
355     if not storage.exists(report_uri, config=sc):
356         return None
357     report = _read_bucket_json(report_uri, sc)
358     if not report or report.get("status") != "success":
359         return None
360     if not video_path:
361         # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
362         # carries the worker's --video-uri (P3); older reports fall back to the
363         # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
364         # recover, and the next same-md5 submit re-binds the truthful path via
365         # add_video's upsert.
366         video_path = str(report.get("video_uri") or "") or _first_object_under(
367             f"{output_root}/videos/{video_id}/", sc
368         )
369     play_wm, cand_wm = db.segment_watermarks(video_id)
370     db.add_video(video_id, video_path, "processed", [])
371     n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
372     if n <= 0:
373         return None
374     segments = [
375         s
376         for s in db.query_play_segments(video_id, {})
377         if int(s.get("id", 0)) > play_wm
378     ]
379     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
380     if not segments:
381         return None
382     logger.info(
383         "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
384         n,
385         video_id,
386         outputs_uri,
387     )
388     return _cached_process_payload(video_id, [], segments, requested_compute)
389 except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
390     logger.warning(
391         "[API] submit-time cache check failed for %s ? dispatching normally",
392         video_id,
393         exc_info=True,
394     )
395     return None
396
397
398 def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
399     """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
400     try:
401         return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
402 except Exception: # noqa: BLE001 ? recovery nicety, never a failure
403     return ""
404

```

```

405
406 _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
407
408
409 def read_path_rehydrate(db, cfg, video_id: str) -> bool:
410     """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
411     nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
412     so
413     ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
414     DB
415     has the video's rows now (re-query). Non-md5 ids return False without touching the
416     bucket; every failure degrades to False (the read just reports what the DB has)."""
417     vid = (video_id or "").strip().lower()
418     if not _MD5_HEX_RE.fullmatch(vid):
419         return False
420     if not probe_process_cache(db, cfg, vid):
421         return False
422     return cached_process_result(db, cfg, vid, "") is not None
423
424 def _maybe_dispatch_remote(
425     cfg,
426     db,
427     req: "ProcessRequest",
428     video_id: str,
429     seg_name: str,
430     val_results,
431     play_wm: int,
432     cand_wm: int,
433     notify,
434 ) -> Optional[Tuple[Dict[str, Any], bool]]:
435     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
436     ``deployment.compute_target
437     == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
438     intervals into
439     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
440     Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
441     the cloud job
442     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
443     observability report
444     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
445     process segmenter
446     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
447     non-``cloud_run`` target, so
448     the in-process path stays byte-identical)."""
449     from backend.compute import ComputeJobSpec, get_compute_backend
450
451     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
452         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
453         (id > play_wm);
454         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
455         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
456         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
457         only when the
458         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
459         rc / ingest
460         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
461         records the
462         # intended backend, `requested` the picker choice, `dispatched` whether the
463         remote job ran.
464         return (
465             {
466                 "video_id": video_id,
467                 "status": "failed",

```



```

457         "message": msg,
458         "results": [r.model_dump() for r in val_results],
459         "play_segments": [],
460         "compute": {
461             "path": "cloud_run" if ran else "not_run",
462             "requested": req.compute,
463             "target": "cloud_run",
464             "dispatched": ran,
465         },
466     },
467     ran,
468 )
469
470 # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471 # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472 # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473 choice = (req.compute or "").strip().lower()
474 if choice and choice not in ("local", "cloud"):
475     logger.warning(
476         "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
477         req.compute,
478     )
479     choice = ""
480 if choice == "local":
481     # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482     # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483     # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484     # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485     # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486     # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487     # below exactly as before (the weights-present check is the escape hatch).
488     if (
489         (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490         and _cloud_available(cfg)
491         and not _local_detector_weights_present(cfg, seg_name)
492     ):
493         return _failed(
494             f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495             "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
496             False,
497         )
498     return None # explicit "run on my machine" ? in-process regardless of the
server's target
499 if choice == "cloud":
500     if not _cloud_available(cfg):
501         return _failed(
502             "cloud-serving was requested but this server isn't configured for it "
503             "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504             False,
505         )
506     compute = get_compute_backend(cfg, target_override="cloud_run")
507 else:

```

```

508         compute = get_compute_backend(
509             cfg
510         ) # server default (default-OFF unless cloud_run)
511     if compute is None:
512         return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514     storage_cfg = getattr(cfg, "storage", None)
515
516     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
517     # to this box.
518     try:
519         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
520     except ValueError as e:
521         return _failed(
522             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
523         )
524     if input_ref.backend == "local":
525         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
526         # different machine)
527         # can't read it. Stage it UP to the input bucket first, then dispatch from there
528         (#461). Only
529         # possible when a CLOUD input_root is configured; without one we genuinely can't
530         # stage ? fail.
531         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
532         # URI below is
533         # built by joining under input_root, and storage.put routes by
534         # parse_uri(staged_uri).backend ?
535         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
536         # at its
537         # default, while a bare/local-looking root can't stage no matter what
538         # input_backend says
539         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
540         # an empty root
541         # gives us nothing to build the staged URI under).
542         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
543         try:
544             root_backend = (
545                 storage.parse_uri(input_root).backend if input_root else "local"
546             )
547         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
548             to
549             root_backend = "local"
550         if root_backend == "local":
551             return _failed(
552                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
553                 storage.input_root to "
554                 "stage local uploads into; a path local to this machine can't be read by
555                 the Cloud "
556                 "Run job.",
557                 False,
558             )
559         local_src = input_ref.key # resolved local path of the upload
560         staged_uri =
561         f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
562         try:
563             notify("staging upload to bucket")
564             # Thread the active storage settings so backend-specific config survives (S3
565             endpoint_url/
566             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
567             default client.
568             storage.put(
569                 local_src,
570                 staged_uri,
571                 config=(
572                     storage_cfg.model_dump() if storage_cfg is not None else None

```

```

558         ),
559     )
560     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561         logger.error(
562             "[API] cloud-serving: failed to stage %s -> %s: %s",
563             local_src,
564             staged_uri,
565             e,
566             exc_info=True,
567         )
568         return _failed(
569             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570             False,
571         )
572     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575     req.video_path = staged_uri
576     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577     db.add_video(
578         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579     )
580     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581     if not out_root:
582         return _failed(
583             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584             "? that's where the Cloud Run job writes the result.",
585             False,
586         )
587
588     notify("dispatching (cloud_run)")
589     logger.info(
590         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591         req.video_path,
592         out_root,
593     )
594     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598     overrides = []
599     idx_cfg = getattr(cfg, "indexing", None)
600     sk = getattr(idx_cfg, "skip_ai_handoff", None)
601     if sk is not None:
602         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607     # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend

```

```

608         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609         wasb_cfg = getattr(idx_cfg, "wasb", None)
610         if wasb_cfg is not None:
611             decode_backend = getattr(wasb_cfg, "decode_backend", None)
612             if decode_backend:
613                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614             batch_size = getattr(wasb_cfg, "batch_size", None)
615             if batch_size:
616                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
617             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
618             if prefetch:
619                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
620         spec = ComputeJobSpec(
621             video_uri=req.video_path,
622             out_uri=out_root,
623             segmenter=seg_name,
624             config_overrides=tuple(overrides),
625         )
626         try:
627             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
628             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
629             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
        which
630             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
631             result = compute.run_infer(
632                 spec,
633                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
634             )
635         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
636             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
637             return _failed(f"cloud dispatch error: {e}", True)
638         if result.returncode != 0:
639             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
640         if result.video_id and result.video_id != video_id:
641             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
642             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
643             logger.warning(
644                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
645                 "and the worker hash; ingesting under the local id.",
646                 result.video_id,
647                 video_id,
648             )
649
650         notify("ingesting (cloud_run)")
651         storage_dict = (
652             storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
653             if hasattr(storage_cfg, "model_dump")
654             else {}
655         )
656         try:
657             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
658         except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
659             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
660             return _failed(
661                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
662                 True,
663             )

```

```

664
665     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
666     segments = [
667         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
668     ]
669     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
670     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
671     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
672     provenance = {
673         "path": "cloud_run",
674         "requested": req.compute,
675         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
676         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
677         "execution": result.execution,
678         "outputs_uri": result.outputs_uri,
679     }
680     logger.info(
681         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
682         result.execution,
683         provenance["job"],
684         provenance["region"],
685         result.outputs_uri,
686         video_id,
687     )
688     return (
689         {
690             "video_id": video_id,
691             "status": "success",
692             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
693             "results": [r.model_dump() for r in val_results],
694             "play_segments": segments,
695             "compute": provenance,
696         },
697         True,
698     )
699
700
701 def _execute_processing(
702     config=None, db=None, req=None, progress=None
703 ) -> Dict[str, Any]:
704     """The full hash ? validate ? segment ? report pipeline for one video.
705
706     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
707     module-level ``global_config``/``db_instance`` (backward-compatible with existing
708     tests that pass only ``req``). Route handlers always pass explicit params; older
709     test code that calls ``_execute_processing(req)`` still works.
710
711     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
712     Raises on unexpected internal errors ? the caller records those as a job failure
713     """
714     import backend.main as _main # call-time seam: main's globals/model stay patchable
715
716     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
717     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
718     if config is not None and not isinstance(config, AppConfig):
719         # Old signature: _execute_processing(req, progress=None)
720         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
721             req = db

```

```

722         db = None
723     elif req is None and isinstance(config, _main.ProcessRequest):
724         req = config
725         config = None
726 # Resolve config/db from module globals when not passed explicitly
727 if config is None:
728     config = _main.global_config
729 if db is None:
730     db = _main.db_instance
731
732 notify = progress or (lambda stage: None)
733
734 notify("hashing")
735 # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
736 # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
737 # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
738 # consumers (hash / validators / segmenter) use the resolved local path.
739 local_video = ""
740 try:
741     local_video = storage.acquire_local_input(
742         req.video_path, getattr(config, "storage", None)
743     )
744     video_id = _main.compute_video_id(local_video)
745 except Exception:
746     storage.cleanup_acquired_local_input(
747         req.video_path, local_video, getattr(config, "storage", None)
748     )
749     raise
750 try:
751     existing_video = db.get_video(video_id)
752 except Exception:
753     storage.cleanup_acquired_local_input(
754         req.video_path, local_video, getattr(config, "storage", None)
755     )
756     raise
757 was_reingest = existing_video is not None
758
759 # typed AppConfig: attribute access for the default segmenter (with safe fallback)
760 default_seg = getattr(
761     getattr(config, "indexing", None), "default_segmenter", "motion"
762 )
763 seg_name = req.segmenter or default_seg
764 segmenter_actual = None
765 segmentation_ran = False
766 val_results: List[Any] = []
767 val_context: Dict[str, Any] = {}
768 # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
769 # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
770 # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
771 compute_actual: Optional[str] = None
772 response: Optional[Dict[str, Any]] = None
773 try:
774     if (
775         existing_video
776         and existing_video.get("status") == "processed"
777         and not req.force
778     ):
779         cached_segments = db.query_play_segments(video_id, {})
780         if cached_segments:

```

```

781         logger.info(
782             "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
783             video_id,
784         )
785         response = _cached_process_payload(
786             video_id,
787             existing_video.get("validation_results", []),
788             cached_segments,
789             getattr(req, "compute", None),
790         )
791         return response
792
793     # 1. Run pluggable validators (with-context variant surfaces ffmpeg_meta for
the report)
794     notify("validating")
795     logger.info(f"Running validations for {req.video_path}...")
796     val_results, val_context = registry.run_all_with_context(local_video, config)
797     failures = [r for r in val_results if not r.passed]
798
799     if failures:
800         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
801         # status; it no longer destroys the video's prior good segments.
802         db.add_video(
803             video_id,
804             req.video_path,
805             "failed",
806             [r.model_dump() for r in val_results],
807         )
808         response = {
809             "video_id": video_id,
810             "status": "failed",
811             "message": _validation_failure_message(failures),
812             "results": [r.model_dump() for r in val_results],
813         }
814         return response
815
816     # 2. Run segmenter & indexer
817     logger.info("[API] Video passed checks. Segmenting and indexing...")
818     db.add_video(
819         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
820     )
821
822     segmenter_cls = segmenter_registry.get(seg_name)
823     if not segmenter_cls:
824         # Submit-time validation already 400s unknown names; this is the defensive
825         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
826         available = list(segmenter_registry.list_available().keys())
827         response = {
828             "video_id": video_id,
829             "status": "failed",
830             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
831             "results": [r.model_dump() for r in val_results],
832             "play_segments": [],
833         }
834         return response
835
836     logger.info(f"[API] Using segmenter: {seg_name}")
837     notify(f"segmenting ({seg_name})")
838     segmenter_actual = seg_name
839     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
840     # if the run yields segments; otherwise drop the new partial rows and keep the

```

```

old.
841         play_wm, cand_wm = db.segment_watermarks(video_id)
842         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
843         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
844         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
845         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
846         # they're dropped on the cloud path in this slice ? a documented follow-up.)
847         remote = _maybe_dispatch_remote(
848             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
849         )
850         if remote is not None:
851             response, segmentation_ran = (
852                 remote # `ran` = did the cloud job actually execute (report)
853             )
854             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
855         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
856         compute_actual = "in_process"
857         segmenter = segmenter_cls(db, config)
858         try:
859             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
860             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
861             with _LOCAL_COMPUTE_LANE:
862                 result = segmenter.process_video(
863                     video_id, local_video, req.player_pool, req.max_frames
864                 )
865         except Exception:
866             # A raising segmenter must not leave half-written rows: restore the pre-run
867             # state (same destroy-after-confirm contract as the Failure branch), then
let
868             # the job worker record the failure.
869             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
870             raise
871         segmentation_ran = True
872
873         if isinstance(result, Failure):
874             logger.error(f"[API] Segmenter failed: {result.message}")
875             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
876             response = {
877                 "video_id": video_id,
878                 "status": "failed",
879                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
880                 "results": [r.model_dump() for r in val_results],
881                 "play_segments": [],
882             }
883             return response
884
885         segments = result.value if hasattr(result, "value") else result
886         notify("finalizing")
887         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
888
889         response = {
890             "video_id": video_id,
891             "status": "success",
892             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value
893             "results": [r.model_dump() for r in val_results],

```



```

894         "play_segments": segments,
895     }
896     return response
897 finally:
898     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
899     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
900     # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
901     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
902     if isinstance(response, dict):
903         response.setdefault(
904             "compute",
905             {
906                 "path": compute_actual or "not_run",
907                 "requested": getattr(req, "compute", None),
908             },
909         )
910     prov = response.get("compute", {}) if isinstance(response, dict) else {}
911     logger.info(
912         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
913         video_id,
914         getattr(req, "compute", None),
915         prov.get("path"),
916         prov.get("dispatched"),
917         (response or {}).get("status") if isinstance(response, dict) else None,
918     )
919     # Always emit the per-video observability report (next to the video, or to
920     # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
921     # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
922     # doesn't surface arbitrary run signals.)
923     paths = _main.emit_indexer_report(
924         db=db,
925         video_id=video_id,
926         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
927         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
928         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
929         # still preserved in DB provenance via db.add_video.
930         video_path=local_video,
931         config=_report_config_for_input(config, req.video_path),
932         val_results=val_results,
933         val_context=val_context,
934         segmenter_requested=req.segmenter,
935         segmenter_actual=segmenter_actual,
936         was_reingest=was_reingest,
937         segmentation_ran=segmentation_ran,
938     )
939     if paths and isinstance(response, dict):
940         response["reports"] = paths
941     storage.cleanup_acquired_local_input(
942         req.video_path, local_video, getattr(config, "storage", None)
943     )
944
945
946 def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
947     """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
948
949     Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat

```

```

that needs a
950         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
951         to a no-op rather than blocking a job it cannot measure."""
952         try:
953             sc = (
954                 storage_config.model_dump()
955                 if hasattr(storage_config, "model_dump")
956                 else (storage_config or {})
957             )
958             backend = storage.get_storage(input_ref.backend, config=sc)
959             return int(backend.stat(input_ref).size)
960         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
961             logger.debug(
962                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
963             )
964             return None
965
966
967     class _CompileError(Exception):
968         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
969         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
970
971         def __init__(self, status_code: int, detail: str):
972             super().__init__(detail)
973             self.status_code = status_code
974             self.detail = detail
975
976
977     def _resolve_compile(
978         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
979     ):
980         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
981         row could change between submit and run). Verifies the video exists, the requested
segments match
982         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
983         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
984         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
985         video = db.get_video(video_id)
986         if not video:
987             raise _CompileError(404, "Indexed video record not found.")
988         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
989         try:
990             out_name = safe_output_filename(output_filename)
991         except ValueError as e:
992             raise _CompileError(400, str(e)) from e
993
994         all_segments = db.query_play_segments(video_id, {})
995         matched = [s for s in all_segments if s["id"] in segment_ids]
996         if not matched:
997             raise _CompileError(400, "No matching play segments found to stitch.")
998
999         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1000         kept = []
1001         for s in matched:
1002             meta = s.get("metadata") or {}
1003             sc = meta.get("shot_count") if isinstance(meta, dict) else None

```

```

1004         try:
1005             if sc is not None and int(sc) < stitching.min_shot_count:
1006                 continue
1007         except (TypeError, ValueError):
1008             pass # unparseable count -> treat as unknown, keep
1009         kept.append(s)
1010     if len(kept) < len(matched):
1011         logger.info(
1012             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1013             stitching.min_shot_count,
1014             len(matched) - len(kept),
1015             len(matched),
1016         )
1017     if not kept:
1018         raise _CompileError(
1019             400,
1020             f"All {len(matched)} matching segments fall below "
1021             f"stitching.min_shot_count={stitching.min_shot_count}.",
1022         )
1023     return video, kept, out_name
1024
1025
1026     def reel_object_uri(cfg, video_id: str, name: str) -> str:
1027         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
1028         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
1029         mirror-upload and the /api/highlights redirect:
`<output_root>/highlights/<video_id>/<name>`. """
1030         root = (getattr(cfg, "storage", None), "output_root", "") or "").strip()
1031         if not root or "://" not in root:
1032             return ""
1033         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1034
1035
1036     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1037         """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
1038         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
1039         isn't silently constructed as a fresh default. """
1040         sc = getattr(cfg, "storage", None)
1041         return sc.model_dump() if sc is not None else None
1042
1043
1044     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1045         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1046         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1047         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1048         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24). """
1049         reel_uri = reel_object_uri(cfg, video_id, name)
1050         if not reel_uri:
1051             return None
1052         settings = _storage_settings(cfg)
1053         if storage.exists(reel_uri, config=settings):
1054             return storage.signed_url(reel_uri, config=settings, method="GET")
1055         return None
1056
1057
1058     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:

```

```

1059     """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1060     and return the {status, filepath, download_url} result the SPA polls for. Returns a
1061     ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1062     so the SPA always gets a clean verdict instead of a worker-crash error."""
1063     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1064
1065     notify = progress or (lambda stage: None)
1066     params = json.loads(job.get("params") or "{}")
1067     video_id = job.get("video_id") or ""
1068     segment_ids = params.get("segment_ids") or []
1069     locale = params.get("locale")
1070
1071     notify("resolving")
1072     try:
1073         video, kept, out_name = _resolve_compile(
1074             db, cfg, video_id, segment_ids, params.get("output_filename", ""))
1075     )
1076     except _CompileError as e:
1077         return {"status": "failed", "message": e.detail, "video_id": video_id}
1078
1079     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1080     output_dir = (
1081         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1082     )
1083     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1084     # unknown/absent locale keeps the configured default unchanged.
1085     localized_watermark = localize_watermark(stitching.watermark, locale)
1086     compiler = _main.VideoCompiler(
1087         output_dir,
1088         begin_padding=stitching.begin_padding,
1089         end_padding=stitching.end_padding,
1090         watermark=localized_watermark,
1091     )
1092     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1093
1094     notify("stitching")
1095     local_src = ""
1096     try:
1097         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1098         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1099         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1100         local_src = storage.acquire_local_input(
1101             video["filepath"], getattr(cfg, "storage", None))
1102     )
1103     # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1104     # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1105     with _LOCAL_COMPUTE_LANE:
1106         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1107     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1108         logger.error(
1109             "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True)
1110     )
1111     return {
1112         "status": "failed",
1113         "message": f"Highlight stitching failed: {e}",

```

```

1114         "video_id": video_id,
1115     }
1116     finally:
1117         if local_src:
1118             storage.cleanup_acquired_local_input(
1119                 video["filepath"], local_src, getattr(cfg, "storage", None)
1120             )
1121
1122     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
1123     # URL ? Cloud Run
1124     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
1125     # copy dies when the
1126     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
1127     # (the endpoint
1128     # still falls back to the local file on the same instance).
1129     reel_uri = reel_object_uri(cfg, video_id, out_name)
1130     if reel_uri:
1131         try:
1132             notify("uploading reel")
1133             # Thread the active storage settings so the configured backend (S3
1134             # endpoint/region, GCS
1135             # project/client) is used, not a fresh default.
1136             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1137         except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
1138             # already succeeded
1139             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1140
1141     # `filepath` is server-local (not browser-reachable); also return the download URL
1142     # so the SPA needn't
1143     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1144     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1145     notify("finished")
1146     return {
1147         "status": "success",
1148         "filepath": out_path,
1149         "download_url": download_url,
1150         "video_id": video_id,
1151     }

```

11. assistant (2026-07-07T15:09:25.823Z)

Now let me verify the guardrail's weights-resolution logic against the ACTUAL runner code, since a divergence would make the guardrail over-block or under-block. Let me read the runners, the fusion segmenter, the config models, and the capabilities probe.

12. user (2026-07-07T15:09:29.849Z)

```

=== git status/branch ===
perf/validation-shared-decode-pass
d53851b perf(validators): decode scene_cut/blur/relevance frames in one shared pass
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
=== find relevant files ===
__init__.py
__pycache__
audio_features.py
base.py
court_gate.py
device.py
fusion_features.py
native_wasb_runner.py
person_detector.py
rally_gate.py

```

```

stub_runner.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py
--- segmenters ---
__init__.py
__pycache__
_keyhint.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py
--- config ---
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py

```

13. assistant (2026-07-07T15:09:36.770Z)

The working tree is on a different branch without the fix (snapshots are authoritative). Now let me verify the weights-resolution logic in the guardrail matches the actual runners.

14. user (2026-07-07T15:09:37.235Z)

```

1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8      see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16     * weights / repo / python / scratch come from the environment first
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__vid12}_wasb.csv``
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner

```

```

27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     """
36
37     import logging
38     import os
39     import shutil
40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63     # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
64     # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-
run.
65     _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
66
67     # Probe for the GPU-resident decode dep (#506): a wasb env without PyNvVideoCodec must
fail the
68     # healthcheck, not the paid inference subprocess. (The NVDEC driver libs ? libnvcuid ?
are only
69     # exercised at decoder INIT, so a missing lib still surfaces in the subprocess; this
catches the
70     # common misconfig, a wasb env built without the package.)
71     _PYNVC_PROBE_CODE = "import PyNvVideoCodec"
72
73
74     @dataclass
75     class NativeWasbConfig:
76         """Resolved settings for a Linux-native WASB run.
77
78         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence

```

```

79         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
80         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
81         """
82
83         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
84         python_bin: str = (
85             "python" # the `wasb` conda env python (invoked by path, no activate)
86         )
87         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
88         sport: str = "badminton"
89         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
90         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
91         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
92         keep_frames: bool = (
93             False # retain the decoded frame cache after success (debug only)
94         )
95         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
96         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
97         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
98         stream_video: bool = True
99         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
100         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
101         batch_size: int = 0
102         prefetch_batches: int = 0
103         # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
104         # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
105         # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
106         decode_backend: str = "cpu"
107
108         @classmethod
109         def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
110             if isinstance(idx_cfg, dict):
111                 idx_cfg = IndexingConfig(**idx_cfg)
112             w = idx_cfg.wasb
113             env = os.environ.get
114             # stream_video is tri-state in config: None/absent => native default (stream);
an
115             # explicit True/False still wins (e.g. False for a container cv2 can't seek).
116             sv = w.stream_video
117             return cls(
118                 repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
119                 python_bin=env("WASB_PYTHON") or w.python_bin,
120                 weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
121                 sport=w.sport,
122                 device=env("WASB_DEVICE") or w.device,
123                 scratch_dir=env("WASB_SCRATCH")
or env("RALLY_SCRATCH")
124                 or env("TMPDIR")
125                 or "",
126                 timeout_sec=w.timeout_sec,
127                 keep_frames=w.keep_frames,
128                 stream_video=(True if sv is None else bool(sv)),
129                 batch_size=int(w.batch_size or 0),
130                 prefetch_batches=int(w.prefetch_batches or 0),
131                 decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
132
133             )

```



```

134
135
136     class NativeWasbRunner(DetectorRunner):
137         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
138
139         def __init__(self, cfg: NativeWasbConfig):
140             self.cfg = cfg
141             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
142             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
143             self._cuda_available: Optional[bool] = None
144
145             # --- low-level native exec (the single place tests patch)
-----
146         def _run(
147             self, argv: List[str], cwd: Optional[str] = None
148         ) -> subprocess.CompletedProcess:
149             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
150             return subprocess.run(
151                 argv,
152                 cwd=cwd,
153                 capture_output=True,
154                 text=True,
155                 timeout=(self.cfg.timeout_sec or None),
156             )
157
158             # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
159         def _probe_cuda(self) -> bool:
160             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
161             (missing python / timeout / import error) is treated as 'no CUDA'."""
162             try:
163                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
164             except (FileNotFoundError, subprocess.TimeoutExpired):
165                 return False
166             return res.returncode == 0 and "cuda True" in (res.stdout or "")
167
168         def _cuda_is_available(self) -> bool:
169             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
170             if self._cuda_available is None:
171                 self._cuda_available = self._probe_cuda()
172             return self._cuda_available
173
174         def _effective_device(self) -> str:
175             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
176             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
177             if self.cfg.device == AUTO:
178                 return DeviceContext(AUTO, self._cuda_is_available()).effective
179             return self.cfg.device
180
181         def _repo_src(self) -> str:
182             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
183
184         def _copy_infer_script(self) -> bool:
185             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
186             repo_src = self._repo_src()
187             try:
188                 os.makedirs(repo_src, exist_ok=True)

```

```

189         shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
190         return True
191     except OSError as e:
192         logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
193         return False
194
195     def _rm_rf(self, path: Optional[str]) -> None:
196         """Best-effort recursive delete of a native path (post-success cleanup; never
197         raises)."""
198         if path:
199             shutil.rmtree(path, ignore_errors=True)
200
201     def healthcheck(self) -> Tuple[bool, str]:
202         """Verify weights + repo present and the `wasb` python imports torch (and sees a
203         GPU on cuda)."""
204         if not self.cfg.weights_path:
205             return False, (
206                 "WASB weights path is empty ? set WASB_WEIGHTS (or
207                 indexing.wasb.weights_path).")
208         weights = os.path.expanduser(self.cfg.weights_path)
209         if not os.path.exists(weights):
210             return False, f"WASB weights not found at {weights}"
211         repo_src = self._repo_src()
212         if not os.path.isdir(repo_src):
213             return False, (
214                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
215                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
216         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
217         # argparse rc=2 from wasb_infer.py at run time.
218         if self.cfg.device not in VALID_DEVICES:
219             return False, (
220                 f"unknown device {self.cfg.device!r} ? valid: {'',
221                 ''.join(VALID_DEVICES)}").
222         if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
223             return False, (
224                 f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
225                 f"{'', ''.join(_VALID_DECODE_BACKENDS)}").
226         try:
227             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
228         except FileNotFoundError:
229             return (
230                 False,
231                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
232             )
233         except subprocess.TimeoutExpired:
234             return False, "native torch healthcheck timed out."
235         out = (res.stdout or "").strip()
236         if res.returncode != 0:
237             return (
238                 False,
239                 f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
240                 out).strip()}",
241             )
242         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
243         subprocess).
244         self._cuda_available = "cuda True" in out
245         ctx = DeviceContext(self.cfg.device, self._cuda_available)
246         if ctx.cuda_required_but_missing:
247             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
248             a silent

```

```

246         # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
247         return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
248     if ctx.fell_back:
249         logger.warning(
250             "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
251             "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
252             out,
253         )
254     # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
the
255     # config validator already rejects) cannot run the GPU-resident path ? fail loud
256     # instead of letting wasb_infer.py die mid-run on a paid box.
257     if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
258         return False, (
259             f"decode_backend=nvdec_resident requires CUDA, but the effective device
is "
260             f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
261         )
262     if self.cfg.decode_backend == "nvdec_resident":
263         try:
264             nv = self._run([self.cfg.python_bin, "-c", _PYNVC_PROBE_CODE])
265         except (FileNotFoundError, subprocess.TimeoutExpired):
266             return False, "PyNvVideoCodec probe failed to run (python
missing/timeout)."
267         if nv.returncode != 0:
268             return False, (
269                 f"decode_backend=nvdec_resident but `{_PYNVC_PROBE_CODE}` failed in
"
270                 f"{self.cfg.python_bin!r}: {(nv.stderr or nv.stdout or
''.strip())[-300:]} "
271                 f"? the wasb env needs PyNvVideoCodec (deploy/cloudrun/Dockerfile /
"
272                 f"gpu_setup.sh stage it)."
273             )
274     return True, out
275
276     def run_predict(
277         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
278     ) -> Optional[str]:
279         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
280
281         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
282         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
283         same downstream contract, so the parity comparison is apples-to-apples.
284         """
285         output_dir = os.path.abspath(output_win_dir)
286         os.makedirs(output_dir, exist_ok=True)
287         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
288         norm = str(video_win_path).replace("\\", "/")
289         stem, _ext = os.path.splitext(os.path.basename(norm))
290         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
291         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
292         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
293
294         if not self._copy_infer_script():
295             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
296             return None
297
298         weights = os.path.expanduser(self.cfg.weights_path)
299         argv = [

```

```

300         self.cfg.python_bin,
301         "wasb_infer.py",
302         "--video",
303         os.path.abspath(str(video_win_path)),
304     ]
305 frames_dir: Optional[str] = None
306 if self.cfg.decode_backend == "nvdec_resident":
307     # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
308     # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
309     # --stream-video nor --frames_out_dir applies and there is no frame cache to
310     # clean.
311     argv += ["--decode-backend", self.cfg.decode_backend]
312 elif self.cfg.stream_video:
313     # Fast path: decode in-process, no PNG extraction. Output is bit-identical
314     # to disk.
315     argv.append("--stream-video")
316 else:
317     scratch = self.cfg.scratch_dir or output_dir
318     frames_dir = os.path.join(scratch, f"{key}_frames")
319     argv += ["--frames_out_dir", frames_dir]
320 # M4: resolve device="auto" to the effective device (cuda-if-available-else-
321 # cpu); an
322 # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
323 wasb_infer.py.
324 device = self._effective_device()
325 if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
326     # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on
327     # a
328     # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
329     # subprocess.
330     logger.error(
331         "decode_backend=nvdec_resident requires CUDA, but the effective device
332         is "
333         "%r ? aborting before the WASB subprocess.",
334         device,
335     )
336     return None
337 argv += [
338     "--weights",
339     weights,
340     "--sport",
341     self.cfg.sport,
342     "--device",
343     device,
344     "--out",
345     out_csv,
346 ]
347 # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
348 # its
349 # byte-parity guarantee) is unchanged when the knobs are unset.
350 if self.cfg.batch_size:
351     argv += ["--batch-size", str(self.cfg.batch_size)]
352 if self.cfg.prefetch_batches and self.cfg.decode_backend != "nvdec_resident":
353     # Prefetch overlaps CPU-side batch assembly with GPU compute ? meaningless
354     # for
355     # the GPU-resident decoder (there is no CPU assembly), whose wasb_infer
356     # branch
357     # ignores the flag; keep the argv honest instead of passing a dead knob.
358     argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
359
360 logger.info(
361     "Running native WASB inference (device=%s%s) on %s ...",
362     device,

```

```

353         " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
354         os.path.basename(norm),
355     )
356     try:
357         res = self._run(argv, cwd=self._repo_src())
358     except FileNotFoundError:
359         logger.error(
360             "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
361         )
362         return None
363     except subprocess.TimeoutExpired:
364         logger.error(
365             "WASB native inference timed out after %ss.", self.cfg.timeout_sec
366         )
367         return None
368     if res.returncode != 0:
369         logger.error(
370             "WASB native inference failed:\n%s",
371             (res.stderr or res.stdout or "")[-2000:],
372         )
373         return None
374     if not os.path.exists(out_csv):
375         logger.error(
376             "WASB native finished but expected CSV not found at %s", out_csv
377         )
378         return None
379     logger.info("WASB trajectory CSV: %s", out_csv)
380
381     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
382     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
383     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
384     if not self.cfg.keep_frames and frames_dir:
385         logger.info(
386             "Cache hygiene: removing frame cache for %s "
387             "(set indexing.wasb.keep_frames=true to retain).",
388             key,
389         )
390         self._rm_rf(frames_dir)
391     return out_csv
392
393     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
394     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
395     trajectory_to_action_windows = staticmethod(
396         TrackNetRunner.trajectory_to_action_windows
397     )
398

```

15. user (2026-07-07T15:09:38.398Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6     1. Translate the Windows video path to its WSL-visible form (and optionally
7       stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8     2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9     3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10    4. Convert that trajectory into high-motion "action windows" ? the same
11       candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12       pipeline (AI handoff, DB population) is unchanged.

```

```

13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import csv
18     import logging
19     import os
20     import shlex
21     import subprocess
22     from dataclasses import dataclass
23     from typing import Any, Dict, List, Optional, Tuple
24
25     from backend.config.models import IndexingConfig
26     from backend.pipeline.detectors.base import (
27         DetectorRunner,
28         WslCommandMixin,
29         wsl_tilde_quote,
30     )
31
32     logger = logging.getLogger(__name__)
33
34
35     @dataclass
36     class TrackNetConfig:
37         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
38
39         distro: str = "Ubuntu"
40         repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
41         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
42         conda_env: str = "TrackNetV4"
43         weights_path: str = "" # WSL path to the .h5/.keras weights (REQUIRED)
44         queue_length: int = 5
45         stage_in_wsl: bool = True # copy video into WSL fs first (perf)
46         wsl_stage_dir: str = "~/clips" # where staged videos/outputs live in WSL
47         timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no timeout)
48         keep_frames: bool = False # retain staged video after success (debug only)
49         min_free_gb: float = 0.0 # absolute floor (GB) atop the ~15x-source BLOCK guard (0
= off)
50
51     @classmethod
52     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "TrackNetConfig":
53         if isinstance(idx_cfg, dict):
54             idx_cfg = IndexingConfig(**idx_cfg)
55         tn = idx_cfg.tracknet
56         return cls(
57             distro=tn.wsl_distro,
58             repo_dir=tn.repo_dir,
59             conda_sh=tn.conda_sh,
60             conda_env=tn.conda_env,
61             weights_path=tn.weights_path,
62             queue_length=tn.queue_length,
63             stage_in_wsl=tn.stage_in_wsl,
64             wsl_stage_dir=tn.wsl_stage_dir,
65             timeout_sec=tn.timeout_sec,
66             keep_frames=tn.keep_frames,
67             min_free_gb=tn.min_free_gb,
68         )
69
70
71     def to_wsl_mnt_path(win_path: str) -> str:
72         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
73
74         Already-POSIX paths (starting with / or ~) are returned unchanged so the
75         same helper is safe to call on either kind of path.

```

```

76     """
77     p = str(win_path)
78     if p.startswith("/") or p.startswith("~"):
79         return p
80     p = p.replace("\\", "/")
81     if len(p) >= 2 and p[1] == ":":
82         drive = p[0].lower()
83         return f"/mnt/{drive}{p[2:]}"
84     return p
85
86
87 @dataclass
88 class TrajectoryPoint:
89     frame: int
90     visible: bool
91     x: float
92     y: float
93
94
95 class TrackNetRunner(WslCommandMixin, DetectorRunner):
96     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
97
98     Implements the common DetectorRunner contract so a future Linux-native or
99     alternative trajectory runner can be substituted without touching the hybrids.
100    """
101
102     def __init__(self, cfg: TrackNetConfig):
103         self.cfg = cfg
104
105     def healthcheck(self) -> Tuple[bool, str]:
106         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
107
108         Returns (ok, message). Cheap to call before a long run.
109         """
110         cmd = (
111             f"conda activate {self.cfg.conda_env} && "
112             f'python -c "import tensorflow as tf; '
113             f'print('TF', tf.__version__, 'GPUs',
114             len(tf.config.list_physical_devices('GPU')))"\n"
115         )
116         try:
117             res = self._wsl_bash(cmd)
118         except FileNotFoundError:
119             return (
120                 False,
121                 "`wsl` not found ? is this running on Windows with WSL2 installed?",
122             )
123         except subprocess.TimeoutExpired:
124             return False, "WSL healthcheck timed out."
125         out = (res.stdout or "").strip()
126         if res.returncode != 0:
127             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
128         return True, out
129
130 # ----- inference
131
132 def run_predict(
133     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
134 ) -> Optional[str]:
135     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
136     None.
137
138     ``output_win_dir`` is a Windows directory (under the repo's output/) that
139     WSL writes into via its /mnt view, so the windows process can read the CSV
140     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and

```

```

139         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
140         a filename cannot collide on the output CSV.
141         """
142         if not self.cfg.weights_path:
143             logger.error(
144                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
145                 "in config.json to the WSL path of your trained TrackNetV4 weights."
146             )
147             return None
148
149         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
150         # relative paths through unchanged, so WSL would resolve them against the
151         # predict.py cwd instead of the repo (same bug class as wasb_runner).
152         output_win_dir = os.path.abspath(output_win_dir)
153         os.makedirs(output_win_dir, exist_ok=True)
154         # Normalize separators so basename/splitext work when tests (or collector)
155         # pass Windows-style paths on a Linux host.
156         video_win_path = str(video_win_path).replace("\\", "/")
157         base = os.path.basename(video_win_path)
158         stem, ext = os.path.splitext(base)
159
160         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
161         if ext.lower() not in (".mp4", ".avi"):
162             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
163             return None
164
165         wsl_out = to_wsl_mnt_path(output_win_dir)
166         # Namespace by video_id so two same-filename videos don't collide on the CSV.
167         # predict.py derives its output name from the (staged) video stem, so staging
168         # under the namespaced name propagates into "<key>_predict.csv".
169         key = f"{stem}_{video_id[:12]}" if video_id else stem
170         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
171
172         # Resolve the video path WSL will read.
173         if self.cfg.stage_in_wsl:
174             staged = self._stage_video(video_win_path, f"{key}{ext}")
175             if staged is None:
176                 return None
177             wsl_video = staged
178         else:
179             # No staging: predict.py reads /mnt directly and writes
180             # "<stem>_predict.csv".
181             # We cannot rename its output, so fall back to the un-namespaced name.
182             wsl_video = to_wsl_mnt_path(video_win_path)
183             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
184
185         # Disk guard (D11): BLOCK before extraction if the WSL stage fs can't hold ~15x
186         # the source
187         # (frame PNGs balloon the footprint). Best-effort measuring ? an undeterminable
188         # read never
189         # blocks; min_free_gb (config) still acts as an absolute floor. See
190         base._wsl_frame_stage_shortfall.
191         shortfall = self._wsl_frame_stage_shortfall(video_win_path)
192         if shortfall:
193             logger.error("TrackNet inference aborted ? %s", shortfall)
194             return None
195
196         cmd = (
197             f"conda activate {self.cfg.conda_env} && "
198             f"cd {self.cfg.repo_dir}/src && "
199             f"python predict.py "
200             f"--video_path {wsl_tilde_quote(wsl_video)} "
201             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
202             f"--output_dir {wsl_tilde_quote(wsl_out)} "
203             f"--queue_length {self.cfg.queue_length}"
204         )

```



```

200         )
201         logger.info("Running TrackNet inference on %s ...", base)
202         try:
203             res = self._wsl_bash(cmd)
204         except subprocess.TimeoutExpired:
205             logger.error(
206                 "TrackNet inference timed out after %ss.", self.cfg.timeout_sec
207             )
208             return None
209
210         if res.returncode != 0:
211             logger.error(
212                 "TrackNet predict.py failed:\n%s", (res.stderr or res.stdout)[-2000:]
213             )
214             return None
215
216         if not os.path.exists(expected_csv_win):
217             logger.error(
218                 "TrackNet finished but expected CSV not found at %s", expected_csv_win
219             )
220             return None
221         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
222
223         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
224         # regenerable intermediate). On earlier failure we return above without
cleaning.
225         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
226             logger.info(
227                 "Cache hygiene: removing staged video for %s "
228                 "(set indexing.tracknet.keep_frames=true to retain).",
229                 key,
230             )
231             self._wsl_rm_rf([wsl_video])
232         return expected_csv_win
233
234     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
235         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
236
237         Returns the WSL path to the staged copy, or None on failure.
238         """
239         src_mnt = to_wsl_mnt_path(video_win_path)
240         reason = self.wsl_source_unreadable_reason(src_mnt)
241         if reason:
242             logger.error("Cannot stage video into WSL: %s", reason)
243             return None
244         dest = f"{self.cfg.wsl_stage_dir}/{base}"
245         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
246         {wsl_tilde_quote(dest)} && echo OK"
247         try:
248             res = self._wsl_bash(cmd)
249         except subprocess.TimeoutExpired:
250             logger.error("Staging video into WSL timed out.")
251             return None
252         if res.returncode != 0 or "OK" not in (res.stdout or ""):
253             logger.error(
254                 "Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip()
255             )
256             return None
257         return dest
258
259     # ----- CSV -> windows
260
261     @staticmethod
262     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
263         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""

```

```

263         points: List[TrajectoryPoint] = []
264         with open(csv_win_path, newline="") as f:
265             reader = csv.DictReader(f)
266             for row in reader:
267                 try:
268                     points.append(
269                         TrajectoryPoint(
270                             frame=int(row["Frame"]),
271                             visible=int(row["Visibility"]) == 1,
272                             x=float(row["X"]),
273                             y=float(row["Y"]),
274                         )
275                     )
276                 except (KeyError, ValueError):
277                     continue
278             points.sort(key=lambda p: p.frame)
279             return points
280
281     @staticmethod
282     def _active_frames(
283         points: List[TrajectoryPoint],
284         fps: float,
285         frame_width: float,
286         velocity_thresh: float,
287     ) -> List[Tuple[int, float]]:
288         """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
289
290         Computes each visible point's normalised velocity from the last *visible* point;
291         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
292         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
293         the caller so identical floats feed the velocity math."""
294         active = [] # (frame_idx, velocity)
295         prev: Optional[TrajectoryPoint] = None
296         for p in points:
297             if not p.visible:
298                 prev = None # break the trajectory; absent shuttle = not in flight
299                 continue
300             if prev is not None:
301                 dframes = p.frame - prev.frame
302                 if dframes > 0:
303                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
304                     velocity = (dist / frame_width) * (fps / dframes)
305                     if velocity > velocity_thresh:
306                         active.append((p.frame, velocity))
307             prev = p
308         return active
309
310     @staticmethod
311     def trajectory_to_action_windows(
312         points: List[TrajectoryPoint],
313         fps: float,
314         frame_width: float,
315         chunk_start: float = 0.0,
316         velocity_thresh: float = 0.02,
317         merge_gap: float = 2.0,
318         min_window_duration: float = 2.0,
319         chunk_index: Optional[int] = None,
320         max_window_duration: float = 0.0,
321         min_split_gap: float = 0.5,
322         window_hard_cap: bool = False,
323         window_inactivity_end: float = 0.0,
324         inactivity_min_density: float = 0.34,

```

```

325         inactivity_rally_thresh: float = 0.08,
326         inactivity_temporal_density: bool = False,
327         window_blob_fallback: bool = False,
328         window_blob_factor: float = 4.0,
329     ) -> List[Dict[str, Any]]:
330         """Convert a shuttle trajectory into candidate rally windows.
331
332         A frame is "active" when the shuttle is visible AND its inter-frame
333         displacement (normalised by frame width, per second) exceeds
334         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
335         ``merge_gap`` seconds of each other are grouped into one window; short
336         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
337         shape feeding the AI-handoff phase is identical.
338
339         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
340         the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
341         and a single window swallows several rallies + the dead time between them. When
342         window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
343         longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
344         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
345         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
346         drop, and it is config-gated default-OFF until measured on the golden set.
347
348         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
349         over-long window has NO internal inactivity gap to split on (a continuously-
active
350         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
351         blob where the gap-splitter alone left a single 174 s window), it is recursively
352         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
353         (never merges), so recall cannot drop; gated default-OFF until measured.
354
355         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
356         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
357         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
358         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
359         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
360         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
361         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
362         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
363         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
364         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
365         """
366         if fps <= 0:
367             fps = 30.0
368         if frame_width <= 0:

```

```

369         frame_width = 1280.0
370
371     active = TrackNetRunner._active_frames(
372         points, fps, frame_width, velocity_thresh
373     )
374
375     if not active:
376         return []
377
378     max_gap_frames = int(merge_gap * fps)
379
380     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
381         vels = [v for _, v in group]
382         return {
383             "start": group[0][0] / fps + chunk_start,
384             "end": group[-1][0] / fps + chunk_start,
385             "active_frame_count": len(group),
386             "peak_velocity": max(vels),
387             "mean_velocity": sum(vels) / len(vels),
388             "track_id_count": 1, # single shuttle; kept for window-shape parity
389             "chunk_index": chunk_index,
390         }
391
392     groups: List[List[Tuple[int, float]]] = []
393     group = [active[0]]
394     for af in active[1:]:
395         if af[0] - group[-1][0] <= max_gap_frames:
396             group.append(af)
397         else:
398             groups.append(group)
399             group = [af]
400     groups.append(group)
401
402     if max_window_duration > 0:
403         groups = TrackNetRunner._split_overlong_groups(
404             groups,
405             int(max_window_duration * fps),
406             int(min_split_gap * fps),
407             hard_cap=window_hard_cap,
408         )
409
410     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
411     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
412     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
413     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
414     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
415     if window_inactivity_end > 0:
416         tw = int(window_inactivity_end * fps)
417         groups = [
418             TrackNetRunner._trim_inactive_tail(
419                 g,
420                 tw,
421                 inactivity_min_density,
422                 inactivity_rally_thresh,
423                 temporal_density=inactivity_temporal_density,
424             )
425             for g in groups
426         ]
427
428     # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4

```

```

trim, any
429         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
430         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
431         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
432         forced_flags = [False] * len(groups)
433         if window_blob_fallback and max_window_duration > 0:
434             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
435                 groups, int(max_window_duration * fps), window_blob_factor
436             )
437
438         windows = []
439         for g, forced in zip(groups, forced_flags):
440             w = _finalize(g)
441             if forced:
442                 w["forced_cut"] = True
443             windows.append(w)
444         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
445
446     @staticmethod
447     def _trim_inactive_tail(
448         group: List[Tuple[int, float]],
449         trail_frames: int,
450         min_density: float,
451         rally_thresh: float,
452         temporal_density: bool = False,
453     ) -> List[Tuple[int, float]]:
454         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
455         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
456         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
457         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
458         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
459         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
460         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
461         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
462         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding).
463
464         ``temporal_density`` (default False = legacy/promoted behaviour) selects the
density
465         DENOMINATOR (code-review finding B). ``group`` carries only active frames, so
the legacy
466         denominator ? the COUNT of active samples in the trailing window (``i - lo +
1``) ? reads a
467         lone fast sample after a long tracking gap as 100% dense and holds the tail
open. When True,
468         the denominator is the trailing window's TEMPORAL length instead, so sparse
tracking can't
469         inflate density. With DENSE tracking the two are identical, so this is a no-op
on
470         well-tracked clips ? hence it ships flag-gated default-OFF until measured on the
golden set
471         at the SERVED operating point (R4 is currently default-ON in prod; see
RALLY_DETECTION_FIXES_PLAN)."""
472         if trail_frames <= 0 or len(group) < 2:
473             return group

```

```

474         frames = [f for f, _ in group]
475         fast = [1 if v > rally_thresh else 0 for _, v in group]
476         lo = 0
477         fast_sum = 0
478         last_dense = -1
479         for i in range(len(frames)):
480             fast_sum += fast[i]
481             while frames[lo] <= frames[i] - trail_frames:
482                 fast_sum -= fast[lo]
483                 lo += 1
484             if temporal_density:
485                 # Trailing window's TEMPORAL length: full ``trail_frames`` once that
much footage
486                 # has elapsed, else the footage seen so far. Sparse tracking can't
inflate it.
487                 denom = min(trail_frames, frames[i] - frames[0] + 1)
488             else:
489                 denom = (
490                     i - lo + 1
491                 ) # legacy: count of active samples in the trailing window
492             if denom > 0 and fast_sum / denom >= min_density:
493                 last_dense = i
494             return group[: last_dense + 1] if last_dense >= 0 else group
495
496     @staticmethod
497     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
498         """Index of the active frame nearest the group's temporal midpoint (used to chop
a
499         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
500         mid = (group[0][0] + group[-1][0]) / 2.0
501         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
502         return split_i
503
504     @staticmethod
505     def _split_overlong_groups(
506         groups: List[List[Tuple[int, float]]],
507         max_win_frames: int,
508         min_split_frames: int,
509         hard_cap: bool = False,
510     ) -> List[List[Tuple[int, float]]]:
511         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
512         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
513         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
514         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
515         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
516         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
517         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
518         if max_win_frames <= 0:
519             return groups
520         out: List[List[Tuple[int, float]]] = []
521         work = list(groups)
522         while work:
523             g = work.pop()
524             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
525                 out.append(g)
526                 continue
527             best_i, best_gap = -1, -1

```

```

528         for i in range(1, len(g)):
529             gap = g[i][0] - g[i - 1][0]
530             if gap > best_gap:
531                 best_gap, best_i = gap, i
532         if best_i < 0 or best_gap < min_split_frames:
533             # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
534             # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
535             if hard_cap:
536                 split_i = TrackNetRunner._midpoint_split_index(g)
537                 work.append(g[:split_i])
538                 work.append(g[split_i:])
539             else:
540                 out.append(g)
541                 continue
542             work.append(g[:best_i])
543             work.append(g[best_i:])
544         out.sort(key=lambda grp: grp[0][0])
545         return out
546
547     @staticmethod
548     def _blob_fallback_chop(
549         groups: List[List[Tuple[int, float]]],
550         max_win_frames: int,
551         blob_factor: float = 4.0,
552     ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
553         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
554         inactivity trim have each had their chance, a group still longer than
`blob_factor` x
555         `max_win_frames` is a genuine continuously-active blob (no internal gap, no
rally-end
556         signal the principled rules could find ? the real-footage `mahadevpura-1` ~65
s residual ?
557         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
558         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
559         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
560         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
561         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
562         can't drop). Returns `(groups, forced_flags)` aligned 1:1; `forced_flags[i]`
is True iff
563         that piece came from a forced chop. Iterative worklist to bound stack depth."""
564         if max_win_frames <= 0:
565             return groups, [False] * len(groups)
566         blob_frames = max_win_frames * max(1.0, blob_factor)
567         out: List[List[Tuple[int, float]]] = []
568         flags: List[bool] = []
569         forced = 0
570         for g in groups:
571             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
572                 out.append(g)
573                 flags.append(False)
574                 continue
575             pieces: List[List[Tuple[int, float]]] = []
576             work = [g]
577             while work:
578                 h = work.pop()
579                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
580                     pieces.append(h)
581                     continue

```

```

582         split_i = TrackNetRunner._midpoint_split_index(h)
583         work.append(h[:split_i])
584         work.append(h[split_i:])
585         pieces.sort(key=lambda grp: grp[0][0])
586         out.extend(pieces)
587         flags.extend([True] * len(pieces))
588         forced += len(pieces) - 1
589     if forced:
590         logger.warning(
591             "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
592             "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
593             "motion), not a bigger cap",
594             forced,
595         )
596     return out, flags
597

```

16. assistant (2026-07-07T15:10:02.026Z)

Now let me verify the fusion substrate logic and config schema, plus how the hybrids resolve the runner (to confirm no hidden default weights path bypasses the guardrail).

17. user (2026-07-07T15:10:03.537Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26    experiment harness (EXPERIMENT_HARNESS.md), never silently.
27    """
28
29    from typing import Any, Dict, List, Optional, Tuple

```



```

30
31 from backend.config.models import FusionConfig, IndexingConfig
32 from backend.pipeline.detectors import fusion_features as ff
33 from backend.pipeline.detectors import rally_gate
34 from backend.pipeline.detectors.base import DetectorRunner
35 from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
36 from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
37 from backend.pipeline.segmenters.base import segmenter_registry
38 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
39
40
41 class FusionSegmenter(TrajectoryHybridSegmenter):
42     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
43 B)."""
44     family = "fusion"
45     display_label = "Fusion"
46
47     def __init__(self, db: Any, config: Any):
48         super().__init__(db, config)
49         # Built lazily only if the person cue is enabled (no detector model loaded
50 otherwise).
51         self._person: Optional[Any] = None
52         # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
53 estimate it
54         # over the whole trajectory once per candidate window (it depends only on
55 `points`).
56         self._axis_cache_key: Optional[int] = None
57         self._axis_cache: Optional[tuple] = None
58
59     @property
60     def name(self) -> str:
61         return "fusion_hybrid"
62
63     @property
64     def description(self) -> str:
65         return (
66             "Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
67 rallies, "
68             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and
69 the "
70             "AI provider sets semantic boundaries."
71         )
72
73     def _build_runner(
74         self, idx_cfg: IndexingConfig
75     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
76         if isinstance(idx_cfg, dict):
77             idx_cfg = IndexingConfig(**idx_cfg)
78         substrate = idx_cfg.fusion.substrate
79         if substrate == "tracknet":
80             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
81             return TrackNetRunner(cfg), {
82                 "weights_path": cfg.weights_path,
83                 "queue_length": cfg.queue_length,
84                 "fusion_substrate": "tracknet",
85             }
86         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
87         return WasbRunner(wcfg), {
88             "weights_path": wcfg.weights_path,
89             "fusion_substrate": "wasb",
90         }
91
92     def _build_native_runner(
93         self, idx_cfg: IndexingConfig
94     ):

```

```

89         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
90             # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
91             # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
92             substrate = idx_cfg.fusion.substrate
93             if substrate != "wasb":
94                 raise NotImplementedError(
95                     "detector_impl='native' supports the WASB substrate only ? set "
96                     "indexing.fusion.substrate='wasb' (no native TrackNet runner yet)."
97                 )
98             from backend.pipeline.detectors.native_wasb_runner import (
99                 NativeWasbConfig,
100                 NativeWasbRunner,
101             )
102
103             cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
104             return NativeWasbRunner(cfg), {
105                 "weights_path": cfg.weights_path,
106                 "detector_impl": "native",
107                 "fusion_substrate": "wasb",
108                 "device": cfg.device,
109                 # Audit the decode path alongside the CPU/nvdec-isolated cache (#506).
110                 "decode_backend": cfg.decode_backend,
111             }
112
113     def _enrich_candidate_stats(
114         self,
115         cw: Dict[str, Any],
116         points: List[Any],
117         fps: float,
118         frame_width: float,
119         video_path: str,
120         idx_cfg: IndexingConfig,
121     ) -> Dict[str, Any]:
122         stats = super()._enrich_candidate_stats(
123             cw, points, fps, frame_width, video_path, idx_cfg
124         )
125         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
126         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
127         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
128         gated = rally_gate.gate_windows(
129             points,
130             [(cw["start"], cw["end"])],
131             fps,
132             min_crossings=0,
133             estimate=self._axis_estimate(points),
134         )
135         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
136
137         fcfg = idx_cfg.fusion
138         if fcfg.cue_flags.get("person", False):
139             stats.update(
140                 self._person_features(cw, points, fps, frame_width, video_path, fcfg)
141             )
142         return stats
143
144     # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
145     # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
146     # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation

```

```

(compute only
147         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
148         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
149         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
150         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
151
152         def _axis_estimate(self, points: List[Any]) -> tuple:
153             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
154             computed once per video, not once per candidate window (it depends only on
points)."""
155             key = id(points)
156             if self._axis_cache_key != key or self._axis_cache is None:
157                 self._axis_cache_key = key
158                 self._axis_cache = rally_gate.estimate_play_axis(points)
159             est = self._axis_cache
160             assert est is not None
161             return est
162
163         def _person_features(
164             self,
165             cw: Dict[str, Any],
166             points: List[Any],
167             fps: float,
168             frame_width: float,
169             video_path: str,
170             fcfg: "FusionConfig",
171         ) -> Dict[str, float]:
172             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
173             frame range and compute the scale/translation-invariant person features. Lazily
builds
174             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
175             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
176             from backend.pipeline.detectors.person_detector import PersonDetector
177
178             if self._person is None:
179                 self._person = PersonDetector(self.config)
180
181             frame_skip = self.config.indexing.yolo_frame_skip
182             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
183             # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
184             start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
185             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
186             fw = det["frame_width"] or frame_width
187             fh = det["frame_height"]
188             # If we can't trust the frame dims, every normalised feature would silently
collapse
189             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
190             if fw <= 0 or fh <= 0:
191                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
192
193             poly = fcfg.court_polygon
194             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
195             net = (fcfg.net_axis, float(fcfg.net_position))
196             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
197             return ff.compute_person_features(
198                 det["frames"], shuttle_in, fw, fh, court_polygon=polygon, net=net

```

```

199         )
200
201     def _select_for_handoff(
202         self,
203         windows: List[Dict[str, Any]],
204         window_stats: List[Dict[str, Any]],
205         idx_cfg: IndexingConfig,
206     ) -> List[int]:
207         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
208         (0) keep every window ? identical to the substrate. Persisted candidates are
209         untouched (full superset).
210
211         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
212         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
213         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
214         set rather than emit a video with zero rallies. This makes Gate A/B strictly
215         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
216         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
217         below it always wins on an otherwise-empty video. Observed on real footage
218         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
219         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
220         fcfg = idx_cfg.fusion
221         min_crossings = int(fcfg.min_crossings)
222         min_players = int(fcfg.min_players)
223         person_on = bool(fcfg.cue_flags.get("person", False))
224
225         keep: List[int] = []
226         for i, st in enumerate(window_stats):
227             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
228                 continue
229             if (
230                 person_on
231                 and min_players > 0
232                 and st.get("players_in_court", 0.0) < min_players
233             ):
234                 continue
235             keep.append(i)
236
237         if not keep and window_stats:
238             # Gating emptied a non-empty video ? fall back to the ungated set.
239             return list(range(len(window_stats)))
240         return keep
241
242
243     segmenter_registry.register(FusionSegmenter)
244

```

18. user (2026-07-07T15:10:04.459Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9

```

```

10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family`` ? config section under ``indexing.*``, output subdir, and
12       the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16       detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20
21     import concurrent.futures
22     import logging
23     import os
24     import time
25     from typing import Any, Dict, List, Optional, Tuple, Union
26
27     import cv2
28
29     from backend.config.models import IndexingConfig
30     from backend.pipeline.detectors.base import DetectorRunner
31     from backend.pipeline.segmenters.base import BasePlaySegmenter
32     from backend.providers import provider_registry
33
34     # Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
35     from backend.results import Failure
36     from backend.sports import sport_registry
37     from backend.utils.paths import output_base
38     from backend.utils.video import extract_video_chunk, get_video_duration
39
40     logger = logging.getLogger(__name__)
41
42
43     def _fmt_ts(seconds: float) -> str:
44         seconds = max(0, int(seconds))
45         h, rem = divmod(seconds, 3600)
46         m, s = divmod(rem, 60)
47         return f"{h:02d}:{m:02d}:{s:02d}"
48
49
50     class TrajectoryHybridSegmenter(BasePlaySegmenter):
51         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
52         boundaries."""
53
54         #: config section under `indexing` (velocity_threshold lives there), the
55         #: output subdir for trajectories, and the metadata detector-tag prefix.
56         family: str = ""
57         #: human-readable label used in log lines.
58         display_label: str = ""
59
60         def _build_runner(
61             self, idx_cfg: IndexingConfig
62         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
63             """Return (runner, detector-specific detection_params extras) for the default
64             (WSL) path."""
65             raise NotImplementedError
66
67         def _build_native_runner(
68             self, idx_cfg: IndexingConfig
69         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
70             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
71             with a
72             native runner override this; the base refuses with a clear message (M3.5: WASB
73             only)."""
74             raise NotImplementedError(

```

```

71         f"detector_impl='native' is not supported for the "
72         f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
73         f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
74     )
75
76     def _resolve_runner(
77         self, idx_cfg: IndexingConfig
78     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
79         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
80
81         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
82         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
83         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
84         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
85         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
86         (the WSL runner) so the default production path is unchanged
87         (``docs/DECOUPLED_COMPUTE/``).
88         """
89         # Safety net: accept raw dicts from legacy / test callers.
90         if isinstance(idx_cfg, dict):
91             idx_cfg = IndexingConfig(**idx_cfg)
92         impl = (
93             os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
94         ).lower()
95         if impl == "stub":
96             from backend.pipeline.detectors.stub_runner import (
97                 StubConfig,
98                 StubDetectorRunner,
99             )
100
101             logger.info(
102                 "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
103                 self.display_label or "trajectory",
104             )
105             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
106                 "detector_impl": "stub"
107             }
108         if impl in ("native", "native_wasb", "wasb_native"):
109             logger.info(
110                 "%s: detector_impl=native ? Linux-native runner (no WSL).",
111                 self.display_label or "trajectory",
112             )
113             return self._build_native_runner(idx_cfg)
114         return self._build_runner(idx_cfg)
115
116     @staticmethod
117     def _probe_fps_width(video_path: str) -> tuple:
118         """Return (fps, frame_width) for a video, with sensible fallbacks."""
119         cap = cv2.VideoCapture(video_path)
120         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
121         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
122         cap.release()
123         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
124
125     @staticmethod
126     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
127         """Provider-reported exchange count, else a duration-based estimate.
128
129         One canonical implementation ? the twins had drifted (wasb relied on

```

```

130         ``int(None)`` raising into the fallback; same result, by accident).
131         """
132         shot_count = segment.get("shuttle_exchange_count")
133         if shot_count is None:
134             return max(3, int(duration / 0.8))
135         try:
136             return int(shot_count)
137         except (ValueError, TypeError):
138             return max(3, int(duration / 0.8))
139
140     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
141     -----
142     def _enrich_candidate_stats(
143         self,
144         cw: Dict[str, Any],
145         points: List[Any],
146         fps: float,
147         frame_width: float,
148         video_path: str,
149         idx_cfg: "IndexingConfig",
150     ) -> Dict[str, Any]:
151         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
152         BASE
153         returns exactly the 4 motion keys, so the trajectory twins persist byte-
154         identical
155         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
156         features). ``points`` are the whole-video trajectory points
157         (TrajectoryPoint)."""
158         return {
159             "peak_velocity": cw["peak_velocity"],
160             "mean_velocity": cw["mean_velocity"],
161             "active_frame_count": cw["active_frame_count"],
162             "track_id_count": cw["track_id_count"],
163         }
164
165     def _select_for_handoff(
166         self,
167         windows: List[Dict[str, Any]],
168         window_stats: List[Dict[str, Any]],
169         idx_cfg: "IndexingConfig",
170     ) -> List[int]:
171         """Indices of the candidate windows that proceed to the AI handoff (and thus may
172         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
173         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
174         Persisted candidates are NOT affected ? they remain the full superset for
175         offline
176         re-scoring."""
177         return list(range(len(windows)))
178
179     def process_video( # type: ignore[override]
180         self,
181         video_id: str,
182         video_path: str,
183         player_pool: Optional[List[str]] = None,
184         max_frames: Optional[int] = None,
185     ) -> "Union[List[Dict[str, Any]], Failure]":
186         # Result contract (the ABC declares Success|Failure). Anything that means the
187         run could
188         # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
189         API key),
190         # an unreadable video, an unavailable detector (native-not-supported), the env
191         healthcheck
192         # not ready, or run_predict timing out / aborting with no trajectory ? returns a
193         ``Failure``
194         # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match

```

```

(a trajectory
186         # was produced but no candidate/AI-confirmed rallies) still returns a bare
187         ``[]``. Every
188         # caller (worker.cmd_infer, main.py, orchestration) branches on
189         isinstance(result, Failure).
190         label = self.display_label
191         # --- Sport profile + AI provider wiring (identical across segmenters) ---
192         sport_name = self.config.sport
193         try:
194             sport_profile = sport_registry.get(sport_name)
195         except KeyError as e:
196             logger.error(str(e))
197             # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
198             return Failure(str(e))
199
200         idx_cfg = self.config.indexing
201         skip_ai = bool(idx_cfg.skip_ai_handoff)
202         provider_name = self.config.ai_provider
203         if skip_ai:
204             model_name = "local-noai"
205             logger.info(
206                 "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
207                 "be emitted as rallies; no %s handoff, no API key needed.",
208                 label,
209                 provider_name,
210             )
211         else:
212             model_name = self.config.ai_model
213             api_key_env_var = f"{provider_name.upper()}_API_KEY"
214             api_key = os.environ.get(api_key_env_var)
215             if not api_key:
216                 from backend.pipeline.segmenters._keyhint import api_key_hint
217
218                 msg = (
219                     f"{api_key_env_var} environment variable is not set! "
220                     f"To run the {provider_name} handoff, set your API key. "
221                     + api_key_hint(api_key_env_var)
222                 )
223                 logger.error(msg)
224                 # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
225                 # marked failed instead of a silent empty "success" (mirrors gemini; the
226                 # own 0-segment diagnostic names this missing-key case as a top cause).
227                 return Failure(msg, is_recoverable=True)
228             try:
229                 provider = provider_registry.get(
230                     provider_name, api_key=api_key, model_name=model_name
231                 )
232             except KeyError as e:
233                 logger.error(str(e))
234                 return Failure(str(e))
235
236         video_duration = get_video_duration(video_path)
237         if video_duration <= 0:
238             logger.error("Invalid video duration.")
239             # An unreadable/broken video, not a 0-rally match.
240             return Failure(
241                 message=f"{label}: invalid/unreadable video duration
242                 ({video_duration}).",
243             )
244         fps, frame_width = self._probe_fps_width(video_path)
245
246         # --- Runner config + windowing thresholds ---
247         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /

```



```

245         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
246         # it delegates to the subclass _build_runner (the real WSL/native runner).
247         try:
248             runner, runner_params = self._resolve_runner(idx_cfg)
249         except NotImplementedError as e:
250             # e.g. detector_impl="native" for a family without a native runner
251             (tracknet, or
252             # fusion with a tracknet substrate). An unavailable detector is a failure,
253             not a
254             # 0-rally match ? refuse cleanly as a Failure instead of crashing or a false
255             success.
256             logger.error("%s: %s", label, e)
257             return Failure(message=f"{label}: {e}")
258
259         velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
260         merge_gap = idx_cfg.dead_time_merge_gap
261         min_window_duration = idx_cfg.min_action_window_duration
262         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
263         IndexingConfig.
264         max_window_duration = idx_cfg.max_window_duration
265         window_min_split_gap = idx_cfg.window_min_split_gap
266         window_hard_cap = idx_cfg.window_hard_cap
267         window_inactivity_end = idx_cfg.window_inactivity_end
268         inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
269         inactivity_min_density = idx_cfg.inactivity_min_density
270         inactivity_temporal_density = idx_cfg.inactivity_temporal_density
271         window_blob_fallback = idx_cfg.window_blob_fallback
272         window_blob_factor = idx_cfg.window_blob_factor
273         handoff_padding = idx_cfg.ai_handoff_padding
274         ai_max_workers = idx_cfg.ai_max_workers
275         log_candidates = idx_cfg.log_yolo_candidates
276         store_candidates = idx_cfg.store_yolo_candidates
277
278         detection_params = {
279             "detector": self.name,
280             "velocity_thresh": velocity_thresh,
281             "merge_gap": merge_gap,
282             "min_action_window_duration": min_window_duration,
283             **runner_params,
284         }
285
286         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
287         ok, msg = runner.healthcheck()
288         if not ok:
289             logger.error("%s detector environment not ready: %s", label, msg)
290             # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
291             the run
292             # failed/retryable instead of writing an empty-but-"successful" result.
293             return Failure(
294                 message=f"{label} detector environment not ready: {msg}",
295                 is_recoverable=True,
296             )
297         logger.info("%s detector env OK: %s", label, msg)
298
299         # --- Phase 2: trajectory -> candidate rally windows ---
300         # Trajectory detectors process the whole video in one pass (they carry
301         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
302         t_start = time.time()
303         out_dir = os.path.join(output_base(self.config), self.family)
304         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
305         if not csv_path:
306             logger.error("%s produced no trajectory; aborting.", label)
307             # The cold-start / timeout failure mode (e.g. WASB native inference timed
308             out after
309             # timeout_sec, or the detector env yielded nothing). This is a DETECTOR

```

```

failure ? NOT
304         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
305         # rather than a silent 0-segment "success" (the wf294 incident).
306         return Failure(
307             message=(
308                 f"{label} detector produced no trajectory (timed out or aborted) ? "
309                 "detector failure, not a 0-rally video."
310             ),
311             is_recoverable=True,
312         )
313
314     points = runner.parse_trajectory_csv(csv_path)
315     logger.info(
316         "Parsed %d trajectory points (%d visible).",
317         len(points),
318         sum(1 for p in points if p.visible),
319     )
320
321     all_candidate_windows = runner.trajectory_to_action_windows(
322         points,
323         fps=fps,
324         frame_width=frame_width,
325         chunk_start=0.0,
326         velocity_thresh=velocity_thresh,
327         merge_gap=merge_gap,
328         min_window_duration=min_window_duration,
329         chunk_index=0,
330         max_window_duration=max_window_duration,
331         min_split_gap=window_min_split_gap,
332         window_hard_cap=window_hard_cap,
333         window_inactivity_end=window_inactivity_end,
334         inactivity_rally_thresh=inactivity_rally_thresh,
335         inactivity_min_density=inactivity_min_density,
336         inactivity_temporal_density=inactivity_temporal_density,
337         window_blob_fallback=window_blob_fallback,
338         window_blob_factor=window_blob_factor,
339     )
340     logger.info(
341         "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
342         label,
343         time.time() - t_start,
344         len(all_candidate_windows),
345     )
346
347     if log_candidates:
348         for cw in all_candidate_windows:
349             logger.info(
350                 f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
351                 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
352                 f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
353             )
354
355     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
356     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
357     # persistence and the handoff gate below.
358     window_stats = [
359         self._enrich_candidate_stats(
360             cw, points, fps, frame_width, video_path, idx_cfg
361         )
362         for cw in all_candidate_windows
363     ]

```

```

364
365 # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
366 candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
367 if store_candidates:
368     for i, cw in enumerate(all_candidate_windows):
369         candidate_ids[i] = self.db.add_candidate_segment(
370             video_id,
371             detector=self.name,
372             start_time=cw["start"],
373             end_time=cw["end"],
374             chunk_index=cw["chunk_index"],
375             confidence=min(1.0, cw["peak_velocity"]),
376             stats=window_stats[i],
377             detection_params=detection_params,
378         )
379
380 if not all_candidate_windows:
381     return []
382
383 # --- Phase 3: AI provider handoff over padded candidate clips ---
384 # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
385 # candidates above stay the full superset ? only the served play_segments are
gated.
386 handoff_indices = self._select_for_handoff(
387     all_candidate_windows, window_stats, idx_cfg
388 )
389
390 ai_segments: List[dict] = []
391 if skip_ai:
392     # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
393     # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
394     # falls back to the duration-based estimate (no AI exchange count).
395     ai_segments = [
396         {
397             "start_time": all_candidate_windows[wi]["start"],
398             "end_time": all_candidate_windows[wi]["end"],
399             "shuttle_exchange_count": None,
400             "shot_count_confidence": "LocalNoAI",
401             "_candidate_id": candidate_ids[wi],
402         }
403         for wi in handoff_indices
404     ]
405     logger.info(
406         "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
407         "(no AI handoff).",
408         len(ai_segments),
409     )
410 else:
411     handoff_dir = os.path.join(
412         output_base(self.config), f"chunks_{provider_name}_handoff"
413     )
414     os.makedirs(handoff_dir, exist_ok=True)
415     logger.info(
416         "Phase 3: %s validation on %d candidate windows...",
417         provider_name,
418         len(handoff_indices),
419     )
420
421 def process_candidate_window(
422     wi: int, window: Dict[str, Any]
423 ) -> Tuple[List[dict], bool]:

```

```

424         """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
425         verdict (clip extraction failed, or the provider returned
metrics['error'])."""
426         w_start = max(0, window["start"] - handoff_padding)
427         w_end = min(video_duration, window["end"] + handoff_padding)
428         w_dur = w_end - w_start
429         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
430         if not extract_video_chunk(
431             video_path, w_start, w_dur, w_path, reencode=True
432         ):
433             return [], True # extraction failed ? no verdict for this window
434         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
435         segments, metrics = provider.analyze_video(
436             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
437         )
438         # B2: a provider failure returns [] with metrics["error"] set ? that is
439         # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
440         # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
441         errored = bool(metrics.get("error"))
442         if errored:
443             logger.warning(
444                 "[Handoff %d] provider error ? window %d skipped: %s",
445                 wi + 1,
446                 wi,
447                 metrics["error"],
448             )
449         valid = []
450         for seg in segments:
451             try:
452                 seg["start_time"] = float(seg["start_time"]) + w_start
453                 seg["end_time"] = float(seg["end_time"]) + w_start
454                 seg["_candidate_id"] = candidate_ids[wi]
455                 valid.append(seg)
456             except (ValueError, TypeError, KeyError) as e:
457                 logger.warning(
458                     "Dropping segment with unparseable timestamps: %s ? %s",
459                     seg,
460                     e,
461                 )
462         try:
463             os.remove(w_path)
464         except OSError:
465             pass
466         return valid, errored
467
468     with concurrent.futures.ThreadPoolExecutor(
469         max_workers=ai_max_workers
470     ) as executor:
471         futures = [
472             executor.submit(
473                 process_candidate_window, wi, all_candidate_windows[wi]
474             )
475             for wi in handoff_indices
476         ]
477         window_results = [future.result() for future in futures]
478     for valid, _errored in window_results:
479         ai_segments.extend(valid)
480     handoff_failed = sum(1 for _valid, errored in window_results if errored)
481     try:
482         os.rmdir(handoff_dir)
483     except OSError:
484         pass

```

```

485         logger.info(
486             "Phase 3 completed. AI confirmed %d semantic play segments.",
487             len(ai_segments),
488         )
489         # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
490         # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
491         # not a 0-rally match. A single window that RAN and returned no rallies is a
legit empty
492         # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
493         if handoff_indices and handoff_failed == len(handoff_indices):
494             msg = (
495                 f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
496                 "window(s) errored (provider error / clip extraction failed) ?
incomplete "
497                 "run, not a 0-rally match."
498             )
499             logger.error(msg)
500             return Failure(message=msg, is_recoverable=True)
501
502         # --- Phase 4: DB population + candidate linking ---
503         segments_data = []
504         for segment in ai_segments:
505             try:
506                 start = float(segment["start_time"])
507                 end = float(segment["end_time"])
508                 dur = end - start
509                 metadata = {
510                     "shot_count": self._shot_count_from(segment, dur),
511                     "shot_count_confidence": segment.get(
512                         "shot_count_confidence", "Unknown"
513                     ),
514                     "detector": f"{self.family}-hybrid-{model_name}",
515                 }
516                 segment_id = self.db.add_play_segment(
517                     video_id, start, end, None, None, metadata
518                 )
519                 if segment_id:
520                     candidate_id = segment.get("_candidate_id")
521                     if candidate_id is not None:
522                         self.db.link_candidate_to_segment(candidate_id, segment_id)
523                     segments_data.append(
524                         {
525                             "id": segment_id,
526                             "start_time": start,
527                             "end_time": end,
528                             "duration": dur,
529                             "metadata": metadata,
530                         }
531                     )
532             except Exception as e:
533                 logger.warning("Skipping invalid segment struct: %s", e)
534
535         return segments_data
536

```

19. assistant (2026-07-07T15:10:35.491Z)

Let me examine the config schema (attribute names, substrate type/default), the WSL WASB runner resolution, the capabilities probe, and the test fixtures.

20. user (2026-07-07T15:10:39.263Z)

```

=== WasbConfig.from_indexing_cfg in wasb_runner.py ===
38:class WasbConfig:
45:     weights_path: str = (
62:     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "WasbConfig":
71:         weights_path=w.weights_path,
183:         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "

```

21. user (2026-07-07T15:10:40.328Z)

```

15:Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
51:     weights_path: str = ""
69:     """Typed config for the live WASB shuttle substrate (indexing.wasb).
80:     weights_path: str = (
131:     decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
168:class FusionConfig(BaseModel):
179:     # Which trajectory substrate seeds the windows (shuttle stays primary).
180:     substrate: Literal["wasb", "tracknet"] = "wasb"
182:     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
198:     net_axis: Literal["x", "y"] = "y"
203:     default_segmenter: str = "yolo_hybrid"
297:     @field_validator("default_segmenter")
349:     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
421:     backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
432:     input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
468:     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
471:     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""

```

22. assistant (2026-07-07T15:10:53.030Z)

`substrate` is `Literal["wasb", "tracknet"]` ? so mixed-case can't occur via validation. Let me read the config model defaults for the weights paths (Pydantic fields the guardrail reads) and the WSL WASB runner's resolution.

23. user (2026-07-07T15:10:54.517Z)

```

40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )
44
45
46     class TrackNetConfig(BaseModel):
47         wsl_distro: str = "Ubuntu"
48         repo_dir: str = "~/models/TrackNetV4"
49         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50         conda_env: str = "TrackNetV4"
51         weights_path: str = ""
52         queue_length: int = 5
53         stage_in_wsl: bool = True
54         wsl_stage_dir: str = "~/clips"
55         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56         velocity_threshold: float = 0.02
57         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58         # decoded frame cache) are deleted automatically ? they are pure intermediates,
59         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61         keep_frames: bool = False
62         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63         # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64         # 0 = no extra floor (the headroom guard still applies).
65         min_free_gb: float = 0.0
66

```

```

67
68 class WasbIndexConfig(BaseModel):
69     """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72     these knobs actually take effect ? previously this whole block was silently dropped
73     because nested config sections defaulted to extra='ignore'.
74     """
75
76     wsl_distro: str = "Ubuntu"
77     repo_dir: str = "~/models/WASB-SBDT"
78     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79     conda_env: str = "wasb"
80     weights_path: str = (
81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91     device: str = "cuda" # auto | cuda | mps | cpu
92     python_bin: str = (
93         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94     )
95     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98     keep_frames: bool = False
99     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101     # 0 = no extra floor (the headroom guard still applies).
102     min_free_gb: float = 0.0
103     # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104     # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105     # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106     # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107     # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108     # Tri-state:
109     # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110     # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111     # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112     stream_video: Optional[bool] = None
113     # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114     # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115     # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no

```

```

116         # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117         # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118         batch_size: Optional[int] = None
119         # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120         # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121         prefetch_batches: Optional[int] = None
122         # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
123         # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
124         # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive
F.interpolate
125         # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
126         # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7x
127         # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native
(Linux)
128         # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
129         # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
130         # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
131         decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
132
133         @model_validator(mode="after")
134         def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
135             # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
136             # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
137             # healthcheck hard-fails if auto resolves to cpu with this backend on.
138             if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
139                 raise ValueError(
140                     f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
141                     f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
142                 )
143             return self
144
145         @field_validator("batch_size")
146         @classmethod
147         def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
148             # A DataLoader batch must be >=1; reject at config time so a typo can't reach
149             # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
150             # when truthy, so 0 stays "off/parity-default").
151             if v is not None and v < 1:
152                 raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
153             return v
154
155         @field_validator("prefetch_batches")
156         @classmethod
157         def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
158             # 0 = off (the default). Reject negatives: the runner passes any truthy value as
159             # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
160             # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
161             if v is not None and v < 0:
162                 raise ValueError(
163                     f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
164                 )

```



```

165         return v
166
167
168     class FusionConfig(BaseModel):
169         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
170 P2).
171
172         Every default reproduces control behaviour: the fusion segmenter persists the
173         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
174         is
175         true ? the person-cue features, but it does NOT gate the served handoff unless a
176         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
177         mask

```

24. user (2026-07-07T15:10:55.314Z)

```

36
37     @dataclass
38     class WasbConfig:
39         """Resolved settings for a WASB WSL run (built from config.json ->
40 indexing.wasb)."""
41
42         distro: str = "Ubuntu"
43         repo_dir: str = "~/models/WASB-SBDT"
44         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
45         conda_env: str = "wasb"
46         weights_path: str = (
47             "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
48         )
49         sport: str = "badminton"
50         wsl_stage_dir: str = "~/clips"
51         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
52         keep_frames: bool = (
53             False # retain staged video + frame cache after success (debug only)
54         )
55         min_free_gb: float = 0.0 # absolute floor (GB) atop the ~15x-source BLOCK guard (0
56 = off)
57
58         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
59         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
60         # extraction that dominates a long clip. Output is bit-identical to the PNG path
61         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
62         it.
63
64         stream_video: bool = False
65
66
67     @classmethod
68     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "WasbConfig":
69         if isinstance(idx_cfg, dict):
70             idx_cfg = IndexingConfig(**idx_cfg)
71         w = idx_cfg.wasb
72         return cls(
73             distro=w.wsl_distro,
74             repo_dir=w.repo_dir,
75             conda_sh=w.conda_sh,
76             conda_env=w.conda_env,
77             weights_path=w.weights_path,
78             sport=w.sport,
79             wsl_stage_dir=w.wsl_stage_dir,
80             timeout_sec=w.timeout_sec,
81             keep_frames=w.keep_frames,
82             min_free_gb=w.min_free_gb,
83             stream_video=w.stream_video if w.stream_video is not None else False,
84         )
85
86
87

```

25. assistant (2026-07-07T15:12:04.286Z)

The WASB `weights\_path` defaults to a non-empty `~/models/...` path (matching both runners), and `substrate` is a validated Literal. Now let me read the test fixtures (to confirm the tests genuinely exercise the guardrail) and the `/api/capabilities` probe the guardrail claims to mirror.

26. user (2026-07-07T15:12:07.426Z)

```
1      """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3      Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4      intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5      GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6      done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7      dispatches; every other target runs the in-process segmenter byte-identically.
8      """
9
10     import csv
11     import json
12     import os
13
14     import pytest
15
16     pytest.importorskip("httpx")
17     from fastapi.testclient import TestClient
18
19     import backend.compute as compute_pkg
20     import backend.main as m
21     from backend.compute.cloud_run import CloudRunJobClient
22     from backend.config.models import AppConfig
23     from backend.infrastructure.database import Database
24     from backend.infrastructure.execution_policy import ExecutionPolicy
25     from backend.pipeline.validators.base import ValidationResult
26
27     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
28
29
30     def _pass():
31         return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
32
33
34     class _InlineExecutor:
35         def submit(self, fn, *a, **k):
36             fn(*a, **k)
37
38
39     class _FakeCloudClient(CloudRunJobClient):
40         """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
41         into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
42
43         def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44             self.calls: list[list[str]] = []
45             self.video_id, self.returncode = video_id, returncode
46             self.rows = (
47                 rows
48                 if rows is not None
49                 else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
50             )
```

```

51
52     def run_job(self, job_name, argv, *, region, project=None):
53         self.calls.append(list(argv))
54         out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
55         done_uri = argv[argv.index("--done-uri") + 1]
56         if self.returncode == 0:
57             outputs = os.path.join(out_uri, "outputs", self.video_id)
58             os.makedirs(outputs, exist_ok=True)
59             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
60                 json.dump(
61                     {
62                         "video_id": self.video_id,
63                         "segment_count": len(self.rows),
64                         "status": "success",
65                     },
66                     f,
67                 )
68             with open(
69                 os.path.join(outputs, "intervals.csv"),
70                 "w",
71                 newline="",
72                 encoding="utf-8",
73             ) as f:
74                 w = csv.writer(f)
75                 w.writerow(["start", "end", "shot_count", "ending_reason"])
76                 for s, e, sc, er in self.rows:
77                     w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
78             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
79             with open(done_uri, "w", encoding="utf-8") as f:
80                 json.dump(
81                     {
82                         "video_id": self.video_id,
83                         "returncode": self.returncode,
84                         "segment_count": len(self.rows),
85                         "status": "success",
86                     },
87                     f,
88                 )
89             return "exec-1"
90
91     def cancel_execution(self, execution):
92         raise AssertionError(
93             "cancel must never fire on the happy path"
94         ) # pragma: no cover
95
96
97     @pytest.fixture
98     def cloud_env(tmp_path, monkeypatch):
99         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
100         bucket read an
101         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
102         hash+validate."""
103         db = Database(str(tmp_path / "t.db")) # noqa: F841
104         monkeypatch.setattr(m, "db_instance", db)
105         monkeypatch.setattr(m, "db_instance", db)
106         bucket = tmp_path / "bucket"
107         bucket.mkdir()
108         cfg = AppConfig.model_validate(
109             { # noqa: F841
110                 "deployment": {"compute_target": "cloud_run"},
111                 "storage": {"backend": "local", "output_root": str(bucket)},
112             }
113         )
114         monkeypatch.setattr(m, "global_config", cfg)
115         monkeypatch.setattr(m, "global_config", cfg)

```

```

114 monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
115 # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
116 fake_local = tmp_path / "fetched.mp4"
117 fake_local.write_bytes(b"FAKEVIDEOBYTES")
118 monkeypatch.setattr(
119     m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
120 )

```

27. user (2026-07-07T15:12:10.484Z)

```

=== capabilities handler ===
86:from backend.storage.capabilities import (
87:    storage_capabilities, # P3: secret-free "which media can this box use" report
267:    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268:    # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
364:@app.get("/api/capabilities")
365:def capabilities(request: Request):
371:    deployment = getattr(cfg, "deployment", None)
374:    # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
376:    weights_path = (
377:        os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
397:        "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
399:        "deployment": {
400:            "profile": getattr(deployment, "profile", "") or "",
401:            "compute_target": getattr(deployment, "compute_target", "") or "",
408:            "storage": storage_capabilities(
732:                from backend.storage.capabilities import _gdrive_mount_root
766:                "storing into a custom medium is disabled on this hosted deployment "
1237:            ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?

```

28. assistant (2026-07-07T15:12:54.212Z)

Let me read the capabilities handler in full, the stub runner (does it need weights?), the presets, and the rest of the test file.

29. user (2026-07-07T15:12:55.781Z)

```

364 @app.get("/api/capabilities")
365 def capabilities(request: Request):
366     """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
367     it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
368     value. Best-effort + never raises (the wizard degrades gracefully)."""
369     cfg = request.app.state.config
370     indexing = getattr(cfg, "indexing", None)
371     deployment = getattr(cfg, "deployment", None)
372     ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
373     ffprobe = shutil.which(ffprobe_bin())
374     # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
375     wasb = getattr(indexing, "wasb", None)
376     weights_path = (
377         os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
378     )
379     # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
380     # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
381     # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the

```

```

382         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
383         nvidia_smi = shutil.which("nvidia-smi")
384         return {
385             "api_version": API_VERSION,
386             "engine_version": _engine_version(),
387             "segmenters": sorted(segmenter_registry.list_available().keys()),
388             "default_segmenter": getattr(indexing, "default_segmenter", None),
389             "ffmpeg": {
390                 "ffmpeg": bool(ffmpeg),
391                 "ffprobe": bool(ffprobe),
392                 "ffmpeg_path": ffmpeg,
393                 "ffprobe_path": ffprobe,
394             },
395             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
396             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
397             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
398             **uploads_api.upload_caps(cfg), # #480 item 5: the SPA pre-flight reads
max_upload_mb
399             "deployment": {
400                 "profile": getattr(deployment, "profile", "") or "",
401                 "compute_target": getattr(deployment, "compute_target", "") or "",
402                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
403                 # the "run in the cloud" toggle only when true (else it's local-only).
404                 "cloud_available": _cloud_available(cfg),
405             },
406             # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
407             # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
408             "storage": storage_capabilities(
409                 getattr(cfg, "storage", None), local_free_path=output_base(cfg)
410             ),
411         }
412
413
414     def _friendly_video_name(filepath: str) -> str:
415         """A human name for a video from its stored path/URI ? the filename stem
(``~/match_2026.mp4`` ?
416         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
417         Falls back to the whole ``filepath`` if there's no basename."""
418         base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]

```

30. user (2026-07-07T15:12:58.344Z)

```

120         )
121         monkeypatch.setattr(
122             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
123         )
124         return db, bucket, monkeypatch
125
126
127     def _inject_fake(monkeypatch, fake):
128         """Make backend.compute.get_compute_backend (re-imported inside
_underscore_dispatch_remote) build a
129         real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token. ***kw``
130         forwards the per-request ``target_override`` (item 3) so the cloud-picker path
resolves too."""
131         real = compute_pkg.get_compute_backend
132         monkeypatch.setattr(
133             compute_pkg,
134             "get_compute_backend",

```

```

135         lambda config, **kw: real(
136             config, run_client=fake, policy=_FAST, token="tok", **kw
137         ),
138     )
139
140
141 def _meta(row):
142     md = row.get("metadata")
143     return json.loads(md) if isinstance(md, str) else (md or {})
144
145
146 def test_cloud_dispatch_ingests_segments(cloud_env):
147     db, _bucket, monkeypatch = cloud_env
148     fake = _FakeCloudClient()
149     _inject_fake(monkeypatch, fake)
150
151     out = m._execute_processing(
152         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
153     )
154
155     assert out["status"] == "success"
156     assert len(out["play_segments"]) == 2
157     rows = db.query_play_segments(out["video_id"], {})
158     assert len(rows) == 2
159     metas = [_meta(r) for r in rows]
160     assert all(md["source"] == "cloud_run" for md in metas)
161     assert {md.get("shot_count") for md in metas} == {5, 3}
162     assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
163
164
165 def test_cloud_dispatch_builds_correct_spec(cloud_env):
166     _db, bucket, monkeypatch = cloud_env
167     fake = _FakeCloudClient()
168     _inject_fake(monkeypatch, fake)
169
170     m._execute_processing(
171         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
172     )
173
174     assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
175     argv = fake.calls[0]
176     assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
177     assert argv[argv.index("--out-uri") + 1] == str(bucket)
178     assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
179     joined = " ".join(argv)
180     assert (
181         "deployment.compute_target=local_gpu" in joined
182     ) # container runs INLINE (never re-dispatch)
183     assert "indexing.detector_impl=native" in joined
184     assert (
185         "indexing.skip_ai_handoff=false" in joined
186     ) # the engine's AI-handoff stance is forwarded
187
188
189 def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
190     """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
191 it tries the
192 Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
193 is forwarded."""
194     _db, _bucket, monkeypatch = cloud_env
195     m.global_config.indexing.skip_ai_handoff = True
196     fake = _FakeCloudClient()

```

```

=== stub healthcheck ===
4:CSV with **no GPU, no WSL, no model weights** ? so the whole trajectory-hybrid pipeline
84:     def healthcheck(self) -> Tuple[bool, str]:
85:         return True, "stub CPU detector (no GPU/WSL)"
=== presets: detector_impl / compute_target / stub ===
28:COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
30:# Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to warn
31:# when config.json explicitly overrides compute_target into a state inconsistent with the
profile.
34:     "cloud-serving": "cloud_run",
35:     "cpu-mock": "cpu_mock",
39:# that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute_target}``
plus
40:# ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
45:     "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46:     # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
52:     "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
58:     "detector_impl": "native",
75:     "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
77:     "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},

```

32. assistant (2026-07-07T15:14:12.883Z)

Let me confirm `\_InProcSeg` is defined in the test, check the cloud-serving preset's `default\_segmenter`, and see the full preset blocks (to gauge realistic seg_name/detector_impl combos).

33. user (2026-07-07T15:14:19.331Z)

```

42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55             # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56             # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57             "indexing": {
58                 "detector_impl": "native",
59                 "skip_ai_handoff": False,
60                 # GPU-resident NVDEC decode (#506/#507): validated on a real L4 (golden gate
PASS,
61                 # F1 0.574 ? baseline 0.582 @ SERVED) AND on the live Cloud Run L4 (the
served
62                 # trajectory reproduced the VM run ? 26479 ? 26481 visible on GX010128),
then rolled
63                 # out + canaried (DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07). This makes
64                 # nvdec the code-level serving default so it survives an image rebuild; the
worker's
65                 # WASB_DECODE_BACKEND env was the reversible rollout lever. cuda-only
(validator-enforced).
66                 "wasb": {

```

```

67         "batch_size": 64,
68         "prefetch_batches": 4,
69         "decode_backend": "nvdec_resident",
70     },
71 },
72     "storage": {"backend": "gcs"},
73 },
74     "cpu-mock": {
75         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
76         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
77         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
78         "storage": {"backend": "local"},
79     },
80 }
81
82
83 def is_known_profile(profile: str) -> bool:
84     """True for a non-empty, recognised profile name."""
85     return bool(profile) and profile in PROFILE_PRESETS
86

```

34. user (2026-07-07T15:14:19.502Z)

```

=== _InProcSeg / _InProc defs in NEW test ===
384:     class _InProcSeg:
400:         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
455: class _InProcSeg:
483:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
650:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
766:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
787:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
818:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
862:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
884:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
=== does OLD/base test define _InProcSeg? (working tree) ===
384:     class _InProcSeg:
400:         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
455: class _InProcSeg:
483:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
650:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
754:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
776:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

```

35. assistant (2026-07-07T15:16:32.042Z)

I've completed a thorough analysis of the guardrail core logic. Let me do a final verification of the exact control-flow ordering in `\_execute\_processing` to confirm the segmenter is never instantiated before the guardrail returns, and re-confirm the registered names.

36. user (2026-07-07T15:16:38.527Z)

```

=== registered names for wasb/tracknet hybrids ===
backend/pipeline/segmenters/wasb_hybrid.py:30:     def name(self) -> str:
backend/pipeline/segmenters/wasb_hybrid.py:31:         return "wasb_hybrid"
backend/pipeline/segmenters/wasb_hybrid.py:68: segmenter_registry.register(WasbHybridSegmenter)
backend/pipeline/segmenters/tracknet_hybrid.py:28:     def name(self) -> str:
backend/pipeline/segmenters/tracknet_hybrid.py:29:         return "tracknet_hybrid"
backend/pipeline/segmenters/tracknet_hybrid.py:48: segmenter_registry.register(TrackNetHybridSegmenter)
=== confirm guardrail is called BEFORE segmenter instantiation in snapshot ===
423: def _maybe_dispatch_remote(
822:     segmenter_cls = segmenter_registry.get(seg_name)
847:     remote = _maybe_dispatch_remote(

```



```

850:         if remote is not None:
857:             segmenter = segmenter_cls(db, config)
862:             result = segmenter.process_video(

```

37. assistant (2026-07-07T15:17:07.933Z)

My analysis of the guardrail core logic is complete and conclusive. I verified every sub-question in the guardrail-correctness dimension against ground-truth code (the runners, config schema, fusion substrate logic, capabilities probe, presets, and the full control flow in `\_execute\_processing`).

Findings summary (all verified against actual code, not the PR description):

- ****Fires exactly when intended****: `seg\_name.strip().lower() in {wasb\_hybrid, tracknet\_hybrid, fusion\_hybrid}` AND `\_cloud\_available(cfg)` AND `not \_local\_detector\_weights\_present(...)`.
Matches the observed production failure.
- ****No casing/whitespace bypass****: both `req.compute` and `seg\_name` are
`.strip().lower()`-normalized (in the condition and inside `\_local\_detector\_weights\_present`);
`seg\_name` is already a validated registry key by line 822; the frozenset exactly matches the
registered `name` properties (`wasb\_hybrid`/`tracknet\_hybrid`/`fusion\_hybrid`); `substrate` is a
validated `Literal["wasb","tracknet"]`.
- ****Branch placement****: the `choice=="local"` branch always returns (`\_failed` or `None`), so
there is no fallthrough into the `choice=="cloud"` or default dispatch ? an explicit local
request with weights present runs in-process and is never silently dispatched to cloud (test
asserts `fake.calls==[]`).
- ****`\_failed(msg, False)` correct****: prunes only new rows (`play\_after=play\_wm` ? no-op here,
prior segments preserved per destroy-after-confirm); provenance `path=not\_run`,
`requested=local`, `target=cloud\_run`, `dispatched=False`.
- ****No premature instantiation****: `segmenter\_cls(db, config)` is at line 857, after the `remote
is not None` return at 850?854; the weight-loading `__init__` never runs when the guardrail
fires.
- ****Boolean AND****: correct `not` precedence, cheap-first ordering, no side effects, correct
short-circuit.
- ****`finally` block****: preserves the guardrail's `compute` stamp, logs honestly,
`emit\_indexer\_report` never raises, cleanup runs, no row leak.
- ****Weights resolution mirrors the runners****: WASB (`WASB\_WEIGHTS` env ?
`indexing.wasb.weights\_path`, with `expanduser`), TrackNet (`indexing.tracknet.weights\_path`, no
env), fusion-by-substrate ? all match `native\_wasb\_runner`, `wasb\_runner`, `tracknet\_runner`,
and `fusion\_hybrid.\_build\_runner`.

The only theoretical over-fire (a hand-edited `compute\_target=cloud\_run` + `detector\_impl=stub`
combo, which no preset produces and which still leaves the user a working cloud path) is not a
real defect.

No real, concrete defects exist in the guardrail-correctness dimension.

38. user (2026-07-07T15:17:07.974Z)

Structured output provided successfully